

Hierarchical Message-Passing Policies for Multi-Agent Reinforcement Learning

Tommaso Marzi¹, Cesare Alippi^{1,2}, Andrea Cini³

¹ IDSIA USI-SUPSI, Università della Svizzera italiana, Lugano, Switzerland

² Politecnico di Milano, Milan, Italy

³ EPFL, Lausanne, Switzerland

Abstract

Decentralized Multi-Agent Reinforcement Learning (MARL) methods allow for learning scalable multi-agent policies, but suffer from partial observability and induced non-stationarity. These challenges can be addressed by introducing mechanisms that facilitate coordination and high-level planning. Specifically, coordination and temporal abstraction can be achieved through communication (e.g., message passing) and Hierarchical Reinforcement Learning (HRL) approaches to decision-making. However, optimization issues limit the applicability of hierarchical policies to multi-agent systems. As such, the combination of these approaches has not been fully explored. To fill this void, we propose a novel and effective methodology for learning multi-agent hierarchies of message-passing policies. We adopt the feudal HRL framework and rely on a hierarchical graph structure for planning and coordination among agents. Agents at lower levels in the hierarchy receive goals from the upper levels and exchange messages with neighboring agents at the same level. To learn hierarchical multi-agent policies, we design a novel reward-assignment method based on training the lower-level policies to maximize the advantage function associated with the upper levels. Results on relevant benchmarks show that our method performs favorably compared to the state of the art.

1 Introduction

Designing information processing methods to support coordination and planning is a core challenge in Multi-Agent Reinforcement Learning (MARL) [51, 36]. Purely centralized approaches [31] reduce the multi-agent problem to a single-agent formulation and thus cannot scale [85]. Conversely, decentralized methods [6] rely solely on local observations: while being scalable, operating at the level of individual agents can make the environment appear non-stationary as other agents update their policies [34, 65]. The induced non-stationarity, combined with partial observability, can hinder coordination and high-level planning [39]. Similarly, Centralized Training Decentralized Execution (CTDE) approaches [25, 85, 6] train agents on all the available (global) information, but are limited to relying exclusively on local observations at execution time [86, 55]. To overcome these challenges, several methods allow for information sharing among agents. In this regard, the Communication Multi-Agent Deep RL (Comm-MADRL) [87] framework improves coordination and mitigates partial observability by leveraging agent communication [73, 18, 35], possibly during both training and execution [64, 63]. Comm-MADRL approaches often employ graph-based representations [12], which allow for modeling interacting agents as *connected* nodes within a graph structure [21, 22]. In graph-based MARL, local observations are usually processed by relying on weight-sharing message-passing Graph Neural Networks (GNNs) [28, 9, 17]. However, communication on *flat* graphs (i.e., graphs modeling only pairwise interactions) can introduce bottlenecks in information propagation [77, 7], hindering coordination and planning [47, 59]. Conversely, Hierarchical Reinforcement Learning

(HRL) [76, 58, 11, 27] methods organize the learning system into a hierarchical decision-making structure. By doing so, HRL introduces learning biases that facilitate spatiotemporal abstraction and long-term planning [46, 82]. In particular, Feudal Reinforcement Learning (FRL) [19] relies on hierarchies of policies where the upper levels send commands and rewards to lower levels. Combining communication-based approaches with an HRL framework such as FRL has the potential to bring together the best of both worlds, but it is not straightforward. Indeed, FRL requires defining ad-hoc rewards for lower levels. This adds overhead for the designer and hinders end-to-end learning of the hierarchy of policies [78, 59]. Moreover, when a hierarchy is added on top of the base agents, the message-passing mechanisms must be adapted to account for the additional interacting entities.

In this paper, we propose a novel HRL method for communication and coordination among different policies in MARL problems. We do so by adopting the FRL framework and introducing a procedure to avoid the associated reward-shaping and optimization issues. Our approach allows for learning a multi-level hierarchy of feudal message-passing policies, thereby achieving high-level planning and coordination. The hierarchical graph defining the feudal structure is determined according to the current state, which can change over time. To train the resulting hierarchy of policies, we design a novel reward-assignment scheme in which lower-level policies are trained to maximize the advantage function associated with the upper levels. The resulting propagation mechanism is decentralized as each agent optimizes different reward signals that depend on the hierarchical structure at each time step. In particular, we theoretically show that the resulting objective remains aligned with maximizing agents’ local reward. This also makes our method suitable for scenarios that are not fully cooperative (provided a coherent definition of the hierarchical structure). Our hierarchical MARL approach enjoys the same benefits of decentralized communication methods [87, 5], while relying on the feudal paradigm to mitigate non-stationarity and partial observability. Policy optimization can be carried out by applying any policy gradient algorithm to local trajectories at different levels. To summarize, our *contributions* are the following:

1. We introduce a novel method based on FRL for learning multi-level hierarchies of message-passing policies in multi-agent systems.
2. We propose a flexible and adaptive reward-assignment scheme that leverages hierarchical graph structures for learning multi-level feudal policies.
3. We provide theoretical guarantees that the proposed learning scheme generates level-specific reward signals aligned with the global task.

Empirical results show that our method, named *Hierarchical Message-Passing Policy* (HiMPo), achieves strong performance in challenging MARL benchmarks that require coordination among agents. HiMPo does not necessarily require a pre-defined graph structure to operate on, as it can simply rely on the hierarchy for communication. HiMPo is a step toward the design of effective MARL methods for coordination and high-level planning.

2 Preliminaries

Partially-Observable Stochastic Game Multi-agent systems can be formalized as Partially-Observable Stochastic Games (POSGs) [32], i.e., a tuple $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{Z}_i\}_{i \in \mathcal{N}}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \{\mathcal{R}_i\}_{i \in \mathcal{N}}, \gamma \rangle$ where $\mathcal{N}$ is the set of agents, $\mathcal{S}$ is a state space, $\mathcal{Z}_i$ and $\mathcal{A}_i$ are the observation and action spaces of the i -th agent, $\mathcal{P} : \mathcal{S} \times \mathcal{A}_1 \times \dots \times \mathcal{A}_{\mathcal{N}} \rightarrow \mathcal{S}$ is a Markovian transition function depending on the joint set of actions, $\mathcal{R}_i : \mathcal{S} \times \mathcal{A}_1 \times \dots \times \mathcal{A}_{\mathcal{N}} \rightarrow \mathbb{R}$ is the reward function for the i -th agent, and $\gamma \in [0, 1]$ is a discount factor. Local observations $\{\mathcal{z}_i^t\}_{i \in \mathcal{N}}$ are generated as a function of the global state $\mathbf{S}_t$. We consider *homogeneous* POSGs [29, 4], i.e., systems where agents have identical (but individual) observation spaces, action spaces, and reward functions. Agents will then learn a policy with shared parameters [1] that aims to maximize the sum of discounted local rewards. In fully cooperative tasks, agents share the same objective. In mixed (cooperative-competitive) settings, agents’ decisions are also driven by self-interested motives, necessitating local rewards [4].

Hierarchies of Feudal Policies Feudal Reinforcement Learning [19] approaches organize the decision-making structure into a multi-level hierarchy of policies. In FRL, actions taken by lower levels are conditioned on the goals assigned by upper levels, which are also responsible for propagating

reward signals to subordinate agents. In the Feudal Graph Reinforcement Learning [59] paradigm, feudal dependencies are represented with a hierarchical graph $\mathcal{G}^*$, each node corresponding to a level-specific entity. In particular, (sub-)managers are abstract nodes sending goals to their subordinates, while workers represent sub-components of the agent (e.g., actuators). Each level maximizes a separate payoff signal: the top-level manager collects rewards directly from the environment, while lower levels receive an intrinsic reward based on the alignment with received commands.

3 Learning Multi-Agent Hierarchical Message-Passing Policies

Existing architectures for learning hierarchies of feudal policies require designing ad-hoc reward mechanisms, which can result in optimization issues [78, 59]. Applying such methodologies directly to multi-agent systems would suffer from the same drawbacks. Furthermore, in MARL, training the collection of policies by relying on a shared (global) reward neglects the individual contribution of each agent and assumes a fully cooperative scenario. To address these challenges, HiMPo learns a feudal hierarchy of message-passing policies in a decentralized fashion by independently optimizing level-specific signals. In HiMPo, the reward for each agent is defined based on the advantage function of the associated (sub-)manager at the upper level. Unlike previous works on multi-agent HRL [80, 54], agent-specific learning signals are generated by locally evaluating the contribution of each worker or (sub-)manager, rather than in a global fashion. Furthermore, in HiMPo, hierarchical relationships can evolve over time. The following section introduces the proposed methodology. As reference, we consider a 3-level hierarchical graph (refer to Fig. 1, left). After summarizing the decision-making process, we illustrate how to construct the hierarchical graph $\mathcal{G}_t^*$ and use it to assign the level-specific rewards. Then, we provide details on the training routine and theoretical guarantees on the soundness of the proposed reward scheme. We refer the reader to App. A for a schematic overview of the algorithm, in which we divide it into distinct phases that align with Sec. 3.1.

3.1 HiMPo Decision-Making Process

We now detail the decision-making process for a 3-level implementation of HiMPo. At each step t , we represent the policies at different levels through a hierarchical graph $\mathcal{G}_t^*$. Each worker w at the base level corresponds to an agent, i.e., $w \in \mathcal{N}$; the set $\mathcal{N}_\sigma$ of sub-managers s (intermediate level) is derived from the hierarchy (see Sec. 3.2); the top-level manager m is a single node at the top of the hierarchy $\mathcal{G}_t^*$. In Phase 1, information is processed within $\mathcal{G}_t^*$. We denote node-level representations at round l of message passing as $\mathbf{h}_t^{w,l}$ (workers), $\mathbf{h}_t^{s,l}$ (sub-managers), and $\mathbf{h}_t^{m,l}$ (top-level manager). Processing functions can be implemented using, e.g., MLPs. Each worker (agent) w initializes its representations $\mathbf{h}_t^{w,0}$ by encoding the corresponding raw observation $\mathbf{z}_t^w$. Then, the representation $\mathbf{h}_t^{s,0}$ of each sub-manager $s \in \mathcal{N}_\sigma$ is generated by processing and aggregating the representations of its subordinate workers; this workers' partition $\mathcal{C}_t^s$ is derived from $\mathcal{G}_t^*$. Similarly, the representation $\mathbf{h}_t^{m,0}$ of the top-level manager m is initialized based on sub-managers' representations. Since there is a single top-level manager m at the highest level of the hierarchy, we set $\mathbf{h}_t^m \doteq \mathbf{h}_t^{m,0}$, as it has no neighboring nodes to propagate information with. Conversely, workers and sub-managers propagate information through their level-specific graph by performing L_r message-passing rounds [28] as

$$\mathbf{h}_t^{i,l+1} = \text{UPD}_l^i \left(\mathbf{h}_t^{i,l}, \text{AGGR} \left\{ \text{MSG}_l^i(\mathbf{h}_t^{i,l}, \mathbf{h}_t^{j,l}) \right\}_{j \in \mathcal{B}_t(i)} \right), \quad (1)$$

where $\mathcal{B}_t(i)$ denotes the neighbors of the i -th node at time step t , UPD_l^i and MSG_l^i indicate learnable update and message function (e.g., MLPs), and AGGR is a permutation-invariant aggregation function (e.g., the mean). In practice, update and message functions are shared among nodes belonging to the same level. Then, based on such representations, the top-level manager m and each s -th sub-manager send goals $\mathbf{g}_t^{m \rightarrow s} \in \mathbb{R}^{d_\mu}$ and $\mathbf{g}_t^{s \rightarrow w} \in \mathbb{R}^{d_\sigma}$, respectively (Phase 2), and each w -th worker (agent) takes an action $\mathbf{a}_t^w \in \mathcal{A}$ (Phase 3) based on its local observation $\mathbf{o}_t^i$ as

$$\begin{aligned} \mathbf{g}_t^{m \rightarrow s} &= \pi_\mu(\mathbf{o}_t^{m \rightarrow s}), & \mathbf{o}_t^{m \rightarrow s} &= \mathbf{h}_t^m || \mathbf{h}_t^{s,0} || \mathbf{h}_t^{s,L_r} & (\text{manager}) \\ \mathbf{g}_t^{s \rightarrow w} &= \pi_\sigma(\mathbf{o}_t^{s \rightarrow w}), & \mathbf{o}_t^{s \rightarrow w} &= \mathbf{g}_t^{m \rightarrow s} || \mathbf{h}_t^{s,L_r} || \mathbf{h}_t^{w,0} || \mathbf{h}_t^{w,L_r} & (\text{sub-manager}) \\ \mathbf{a}_t^w &= \pi_\omega(\mathbf{o}_t^w), & \mathbf{o}_t^w &= \mathbf{g}_t^{s \rightarrow w} || \mathbf{h}_t^{w,0} || \mathbf{h}_t^{w,L_r} & (\text{worker}) \end{aligned} \quad (2)$$

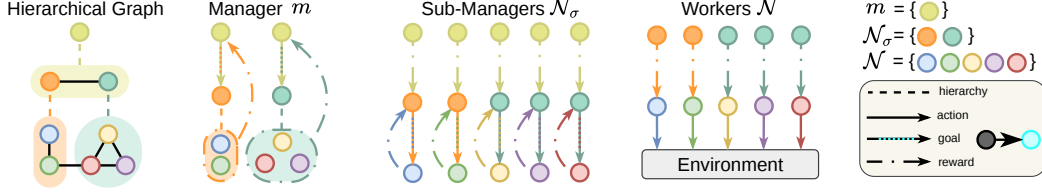


Figure 1: Assignment schemes of rewards and goals in a 3-level hierarchical graph (up to temporal scales). Dashed lines indicate hierarchical relationships; solid arrows denote worker actions; solid arrows with colored dots indicate assigned goals; dash-dot arrows represent rewards.

where $\pi_\mu, \pi_\sigma, \pi_\omega$ indicate the policies of the manager, sub-managers, and workers, respectively. We remark that policies are shared across nodes at the same level, i.e., we learn a single level-specific policy for all the workers (π_ω) and sub-managers (π_σ). Each policy effectively acts in a (learned) latent Partially-Observable Markov Decision Process (POMDP) [40], with observations and actions as in Eq. 2 and rewards defined in the next subsection. As such, for each node $i \in \mathcal{G}_t^*$ with level-specific policy $\pi_\square$, we can define trajectories τ_i , value $V^{\pi_\square}(\mathbf{o}_t^i)$ and advantage $A^{\pi_\square}(\mathbf{o}_t^i, \pi_\square(\mathbf{o}_t^i))$ functions associated with observations and actions (goals). Upper levels operate at different time scales: we assume that the top-level manager and sub-managers send goals every α_μ and α_σ environment step, respectively. We maintain temporal consistency within the hierarchy, i.e., $\alpha_\mu \geq \alpha_\sigma \geq 1$, so that high-level goals are propagated less frequently than low-level ones [19]. To synchronize goal updates across levels, we set $\alpha_\mu = K\alpha_\sigma \doteq K\alpha$, with $K \in \mathbb{Z}^+$. Similar to other works on HRL [78, 80], K and α are hyperparameters that can be tuned according to the problem.

3.2 Building the Hierarchical Graph

The graph $\mathcal{G}_t^*$ encoding the feudal hierarchy is built according to the environment structure. Following the Markovian assumption, it can be defined as a function of the global state $\mathbf{S}_t$. In a 3-level hierarchy, the set $\mathcal{N}_\sigma$ of sub-managers can be dynamically learned [52] or defined based on prior knowledge [59]. We refer the reader to App. D.1.2 for a visual example where we assign each sub-manager to a portion of the state space. All sub-managers are trivially connected to a single top-level manager, while the feudal relationships between workers and sub-managers can be derived from a subset of the observation features (e.g., agents' locations) or structural constraints (e.g., communication channels). We found agents' positions to provide a good enough bias to generate the hierarchical and relational structure without requiring additional knowledge on the environment (see Sec. 5). For the upper levels of the hierarchy (sub-managers), we use fully-connected graphs to exchange messages, although more sophisticated solutions can be explored. Note that the proposed methodology makes no assumptions about the graph used for message passing or the structure of the hierarchy. When no prior knowledge exists, a viable approach is to connect all the workers directly to a single top-level manager, forming a static 2-level hierarchy suitable for problems with no clear intermediate clustering (see Sec. 5.1). Hierarchical relationships can also be learned end-to-end [48, 22], e.g., by relying on graph pooling operators [30]. Although more general, this approach introduces overheads in both computational and sample complexity. To keep the scope of the paper contained, we do not explore this direction.

3.3 Hierarchical Assignment of Rewards

In HiMPo, (sub-)managers operate at different time scales, and the hierarchical graph $\mathcal{G}_t^*$ can be dynamic. Rewards must be assigned locally according to the topology of $\mathcal{G}_t^*$ to avoid sub-optimal solutions. If misspecified, rewards for intermediate levels can lead to undesirable behaviours, such as sub-managers competing against each other. Fig. 1 summarizes the reward-assignment scheme.

Manager Each goal sent by the top-level manager is assigned a local learning signal, based on the rewards obtained by the subset of workers supervised by the sub-manager receiving the goal. In particular, the reward associated with the goal $\mathbf{g}_t^{m \rightarrow s}$ sent by the manager m to the s -th sub-manager at time step t reads:

$$r_t^{m \rightarrow s} = \text{MEAN}_{w \in \mathcal{C}_t^s} \left\{ \sum_{k=0}^{K\alpha-1} \bar{r}_{t+k}^w \right\}, \quad (3)$$

where $\mathcal{C}_t^s$ denotes the partition of workers associated with sub-manager s at step t and $\{\bar{r}_k^w\}_{w=1}^N$ is the set of environment rewards at the k -th time step. In other words, $r_t^{m \rightarrow s}$ is defined as the average cumulative reward obtained by agents in $\mathcal{C}_t^s$ over the $K\alpha$ time steps associated with the goal $\mathbf{g}_t^{m \rightarrow s}$. Notice that, for the duration of goal $\mathbf{g}_t^{m \rightarrow s}$, changes in the subset of workers (and topology $\mathcal{G}_t^*$) are not observed by the manager m . Thus, the cumulative reward $r_t^{m \rightarrow s}$ is computed by considering workers that are subordinate to s at time t , i.e., the step when the goal was sent. We compute average (rather than total) cumulative rewards to account for variations in the number of subordinate workers w.r.t. future time steps. The reward in Eq. 3 allows us to define the manager’s advantage function in the latent POMDP:

$$A^{\pi_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) = \mathbb{E}_{\tau_m} [r_t^{m \rightarrow s} + \gamma V^{\pi_\mu}(\mathbf{o}_{t+K\alpha}^{m \rightarrow s}) - V^{\pi_\mu}(\mathbf{o}_t^{m \rightarrow s})]$$

Sub-managers Learning signals for lower levels must be aligned with the objectives set by the upper levels. We define the reward associated with each sub-manager s according to the performance that manager m achieved by assigning the goal $\mathbf{g}_t^{m \rightarrow s}$ to s . In particular, the reward signals are defined as the manager’s advantage function $A^{\pi_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s})$, scaled by the number K of sub-manager actions over the $K\alpha$ interval. Maximizing the manager’s advantage function implies taking actions that lead to higher returns, encouraging sub-managers to conform to the set goals. Note that $A^{\pi_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s})$ is conditioned on the goal $\mathbf{g}_t^{m \rightarrow s}$ sent by the top-level manager. However, $\mathbf{g}_t^{m \rightarrow s}$ is evaluated by aggregating cumulative rewards among workers in $\mathcal{C}_t^s$. In particular, from the perspective of the sub-manager s , $A^{\pi_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s})$ is the same regardless of the K number of times in which s acts in the $K\alpha$ steps and the number $|\mathcal{C}_t^s|$ of goals sent at each of the step. Thus, we add a local component to the reward associated with each goal to distinguish it from goals assigned to other workers or at different time steps. Specifically, we use the cumulative reward received by worker w over the subsequent α steps. As a result, the reward associated to $\mathbf{g}_t^{s \rightarrow w}$ reads:

$$r_t^{s \rightarrow w} = \frac{A^{\pi_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s})}{K} + \sum_{k=0}^{\alpha-1} \bar{r}_{t+k}^w, \quad (4)$$

and the corresponding sub-manager’s advantage function is:

$$A^{\pi_\sigma}(\mathbf{o}_t^{s \rightarrow w}, \mathbf{g}_t^{s \rightarrow w}) = \mathbb{E}_{\tau_s} [r_t^{s \rightarrow w} + \gamma V^{\pi_\sigma}(\mathbf{o}_{t+\alpha}^{s \rightarrow w}) - V^{\pi_\sigma}(\mathbf{o}_t^{s \rightarrow w})].$$

Workers We define rewards for workers similarly to sub-managers. In particular, each worker w receives a reward consisting of the advantage function $A^{\pi_\sigma}(\mathbf{o}_t^{s \rightarrow w}, \mathbf{g}_t^{s \rightarrow w})$ of the sub-manager s from which the goal $\mathbf{g}_t^{s \rightarrow w}$ has been sent, scaled by the duration α , i.e., as

$$r_t^w = \frac{A^{\pi_\sigma}(\mathbf{o}_t^{s \rightarrow w}, \mathbf{g}_t^{s \rightarrow w})}{\alpha}, \quad (5)$$

where we denote the worker’s reward as r_t^w to distinguish it from the external reward $\bar{r}_t^w$. For each worker w , the reward in Eq. 5 is distributed among the α actions taken in the interval $[t, t + \alpha)$. Note that such rewards are local, i.e., each worker receives a different learning signal. Unlike Eq. 4, there is no need for adding the external reward here, and, as a result, workers do not have direct access to the environment signals. For dynamic hierarchies, we found that using a modified advantage-like definition of $A^{\pi_\sigma}(\mathbf{o}_t^{s \rightarrow w}, \mathbf{g}_t^{s \rightarrow w})$ yields better results for policy optimization (see App. C.4).

End-to-end Training Following the FRL paradigm, each level-specific policy is trained independently from the others to maximize its level-specific return. This allows for generating goals at different spatial and temporal scales, enabling task decomposition within the hierarchy (see Sec. 5). Although level-specific rewards depend on the advantage function at upper levels, ensuring they are aligned with each agent’s objective is a significant challenge: we provide theoretical guarantees on this aspect in Sec. 3.4.

Cooperative and Mixed Settings Note that, provided a proper choice of the hierarchy, HiMPo can be adopted in *both* fully cooperative and self-interested settings: hierarchies with more than 2-levels assume that the environment is cooperative, while a 2-level hierarchy allows for mixed settings. Indeed, in a 3-level hierarchy, the top-level manager aggregates workers’ rewards from the partitions induced by the hierarchical structure $\mathcal{G}^*$ (refer to Eq. 3). Conversely, in 2-level hierarchies, where workers are all connected to a single top-level manager, each worker’s return is maximized separately. In this setting, each worker’s learning signal is directly given by the manager’s advantage function. Note that parameter-sharing can have an impact on the space of learnable multi-agent policies.

3.4 Theoretical Analysis

HiMPo aims to learn a hierarchy of independent policies where each node receives a level-specific reward. To ensure the soundness of the proposed scheme, we must show that each level-specific objective aligns with the global task. Let $T_\mu = \{0, K\alpha, 2K\alpha, \dots, \infty\}$ denote the manager time scale, i.e., the time steps $t \in T_\mu$ at which goals $g_t^{m \rightarrow s}$ are sent. Given joint policy $\pi \doteq (\pi_\omega, \pi_\sigma, \pi_\mu)$, the global objective consists of maximizing the return of each worker w (agent):

$$\eta(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} \sum_{k=0}^{K\alpha-1} \bar{r}_{t+k}^w \right], \quad (6)$$

where τ is a trajectory and $\gamma \doteq \gamma_\mu$ is the discount factor of the top-level manager. The following theoretical results show that maximizing returns obtained from the rewards defined in Sec. 3.3 implies maximizing the return in Eq. 6. In particular, we assume that each level-specific policy is optimized while keeping the others fixed. Proofs for all the theoretical results are provided in App. B.

Theorem 3.1 (Manager). *The maximization of the return $\eta_m(\pi_\mu)$ defined using the rewards in Eq. 3 obtained by manager m sending a goal to sub-manager s implies the maximization of $\eta(\pi)$, i.e.:*

$$\max_{\pi_\mu} \eta_m(\pi_\mu) \doteq \max_{\pi_\mu} \mathbb{E}_{\tau_m \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} r_t^{m \rightarrow s} \right] = \max_{\pi_\mu} \eta(\pi) \quad (7)$$

It follows that the manager’s objective $\eta_m(\pi_\mu)$ is aligned with the global task in Eq. 6.

With a slight abuse of notation, let $\hat{\pi}_\omega$ and $\hat{\pi}_\sigma$ denote joint policies $(\pi_\omega, \tilde{\pi}_\sigma, \tilde{\pi}_\mu)$ and $(\tilde{\pi}_\omega, \pi_\sigma, \tilde{\pi}_\mu)$, where $\tilde{\pi}_{i=\omega, \sigma, \mu}$ is the old version of $\pi_{i=\omega, \sigma, \mu}$. Furthermore, let $T_\sigma = \{0, \alpha, 2\alpha, \dots, \infty\}$ be the sub-managers’ time scale, i.e., the set of time steps t in which goals $g_t^{s \rightarrow w}$ are propagated.

Theorem 3.2 (Sub-managers). *Assuming $\gamma_\sigma \simeq \gamma \simeq 1$ and K not too large, the maximization of the return $\eta_s(\pi_\sigma)$ defined using the rewards in Eq. 4 obtained by the sub-manager s sending a goal to the worker w implies the maximization of $\eta(\hat{\pi}_\sigma)$, i.e.:*

$$\max_{\pi_\sigma} \eta_s(\pi_\sigma) \doteq \max_{\pi_\sigma} \mathbb{E}_{\tau_s \sim \hat{\pi}_\sigma} \left[\sum_{t \in T_\sigma} \gamma_\sigma^{t/\alpha} r_t^{s \rightarrow w} \right] = \max_{\pi_\sigma} \eta(\hat{\pi}_\sigma) \quad (8)$$

It follows that the sub-manager’s objective $\eta_s(\pi_\sigma)$ is aligned with the global objective in Eq. 6.

Theorem 3.3 (Workers). *Assuming $\gamma_\omega \simeq \gamma_\sigma \simeq 1$ and α not too large, the maximization of the return $\eta_w(\pi_\omega)$ defined using the rewards in Eq. 5 obtained by the worker w implies the maximization of $\eta(\hat{\pi}_\omega)$, i.e.:*

$$\max_{\pi_\omega} \eta_w(\pi_\omega) \doteq \max_{\pi_\omega} \mathbb{E}_{\tau_w \sim \hat{\pi}_\omega} \left[\sum_{t=0}^{\infty} \gamma_\omega^t r_t^w \right] = \max_{\pi_\omega} \eta(\hat{\pi}_\omega) \quad (9)$$

It follows that the worker’s objective $\eta_w(\pi_\omega)$ is aligned with the global objective in Eq. 6.

Note that the theoretical results remain valid when using a 2-level hierarchy. In such a case, the learning signal of each worker consists of the manager’s advantage function. In practice, although the theoretical results assume that each policy is optimized while keeping the others fixed, we learn policies across different levels concurrently to improve sample efficiency; previous works empirically showed that this does not compromise performance [50].

4 Related Work

Within communication-based methodologies [87], message-passing architectures have been employed to parameterize policies [1, 64] and enhance global awareness during training and execution without relying on global information [63]. Other works follow a similar approach in the context of multi-robot systems [41, 15, 14]. GNNs have also been used to learn action-value functions in different settings. Jiang et al. [38] leverage an attention mechanism based on the agent graph to learn individual action-value functions, while other works [62, 44] apply GNNs to learn a global value function in fully cooperative scenarios. Other Comm-MADRL approaches either broadcast messages through

static communication structures [25, 74, 18] or directly learn the propagation scheme [73, 37, 49, 35]. Communication has also been leveraged in several HRL approaches [69, 79, 53, 83], but, to the best of our knowledge, none of these methods learn level-specific policies using independent, agent-specific reward signals. Concerning single-agent HRL settings, Li et al. [50] addressed the issue of designing a problem-free reward function for low-level policies by leveraging the advantage function of high-level policies. Subsequently, this reward scheme has been extended to multi-agent systems [80, 54], where architectures are trained under the CTDE paradigm using QMIX [68]. Unlike our approach, here low-level optimization signals are based on the team reward, i.e., actions are not explicitly evaluated according to their local contributions [75]. Moreover, such methods are restricted to 2-level hierarchies. We also note that value factorization [68] enforces global action optimality through specific architectural constraints. This is fundamentally different from our theoretical contribution, which guarantees that the proposed advantage-based hierarchical rewards are aligned with the global task. The FRL [19] framework has been first extended to the deep RL setting [78] and, subsequently, applied to multi-agent systems [2]. In this context, it showed performance improvements across diverse domains, e.g., traffic signal control [56, 57] or warehouse logistics [45]. However, none of these methods adopts a problem-free reward scheme nor considers message-passing policies. Lastly, FRL has been exploited to learn hierarchical graph-based modular policies in single-agent RL [59].

5 Experiments

In the following, we evaluate HiMPo on relevant MARL benchmarks by considering problems of increasing complexity and with a varying number of interacting agents. We use Proximal Policy Optimization (PPO) [72] as the underlying policy optimization algorithm and refer to this specific implementation of HiMPo as Hierarchical Message-Passing PPO (HiMPPO).

Baselines and benchmarks We compare our methodology against state-of-the-art MARL methods. We consider 1) Independent PPO (IPPO) and 2) Multi-Agent PPO (MAPPO), i.e., two MARL variants of PPO proposed by Yu et al. [84]. Concerning graph-based methods, we include in the comparison 3) the homogeneous version of Graph Neural Network PPO (GPPO) [14], 4) H-GPPO, a hierarchical variant of GPPO akin to HAMA [69] that assumes an underlying fully-connected graph, and 5) H-GUPPO, a hierarchical PPO architecture based on Graph U-Nets [26]. Furthermore, in App. C.1 we compare HiMPPO against two additional communication methods that are not based on PPO, i.e., 6) DGN [38] and 7) MAGIC [64]. The main difference of our method with the hierarchical baselines is that the latter use a hierarchical GNN to learn a single weight-sharing policy, while HiMPPO leverages a (dynamic) hierarchical graph to learn level-specific policies and assign rewards. We consider 2 main environments under several different settings: 1) *Level-Based Foraging with Survival* (LBFwS), i.e., a novel version of the LBF [3, 66] environment, to show how the models perform in tasks with challenging temporal credit assignment; 2) the *VMAS Sampling* environment from the VMAS simulator [13], to assess the exploration capabilities of the baselines. In *LBFwS* we consider three levels of increasing difficulty, while for *VMAS Sampling* we test three different team sizes. Additionally, in App. C.5 we report results on 3) *SMACv2* [23], i.e., the improved version of the *StarCraft Multi-Agent Challenge* (SMAC) [70]. For the models using graph-based representations, we use a dynamic graph based on the location of the agents at each time step. In particular, two agents are connected if they are within their observation ranges. Due to the different computational requirements of the environments, for each configuration, we consider 8 independent seeds in *LBFwS*, 6 in *VMAS Sampling*, and 4 in *SMACv2*. Additional details are reported in App. D.

5.1 Level-Based Foraging with Survival (LBFwS)

In *LBFwS*, agents navigate in a grid world where resources of different value are randomly spawned (App. D.1.1). Consuming resources increases the individual reward, while delivering them to a fixed location extends the episode duration. Higher-value items need the cooperation of multiple agents to be obtained. An agent can learn either to simply maximize its reward (individual strategy) or to sacrifice the local reward to extend the episode and possibly achieve a higher final return (cooperative strategy). In particular, we consider a system of 10 agents and three difficulty levels (*Easy*, *Medium*, and *Hard*) that correspond to progressively larger grid sizes and varying amounts of resources (see App. D.1.1). For HiMPPO, we consider a static 2-level hierarchical graph

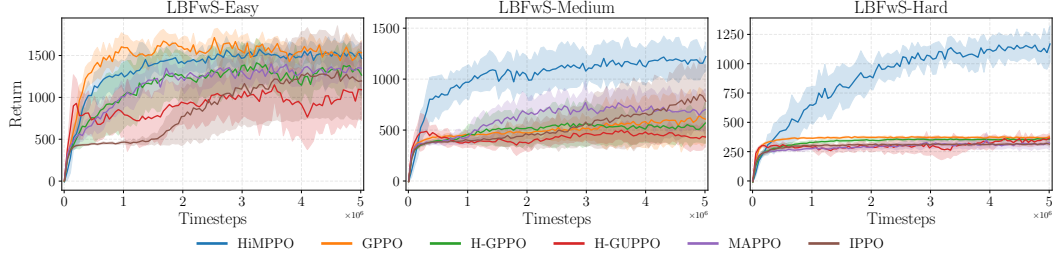


Figure 2: Results of the models for different difficulty levels in the *LBFwS* environment (avg. return $\pm$ std. of 8 runs).

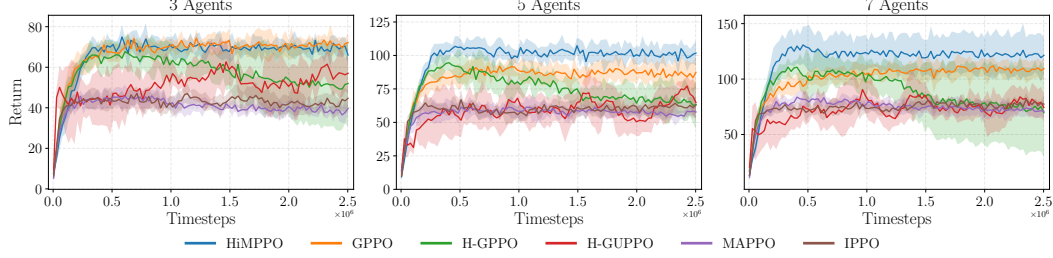


Figure 3: Results of the models for different numbers of agents in the *VMAS Sampling* environment (avg. return $\pm$ std. of 6 runs).

with all the agents (workers) connected to a single top-level manager that sends goals every $K = 5$ steps. We investigate the impact of the goal scale K in App. C.3.

Results are reported in Fig. 2. All the models can achieve high returns in the *LBFwS-Easy* scenario, with HiMPPO and GPPO being more sample efficient. However, as the difficulty increases, the baselines struggle to achieve good results. Notably, in *LBFwS-Hard*, all the methods except HiMPPO learn to maximize the individual reward. Conversely, HiMPPO consistently achieves good results at every difficulty level. These results show the effectiveness of HiMPPO in coordination and planning capabilities. Furthermore, the performance achieved by other hierarchical methods (H-GPPO and H-GUPPO) shows that relying on external reward fails in more complex scenarios, supporting the adoption of our reward-assignment mechanism.

5.2 VMAS Sampling

In this continuous control problem, n robots are randomly spawned in a grid. The grid has an underlying Gaussian density function with 3 modes. Visiting a cell for the first time leads to a reward proportional to the density function. For HiMPPO, the dynamic 3-level hierarchical graph $\mathcal{G}_t^*$ is generated according to the positions of the robots at each step t (refer to App. D.1.2); the goal scales are set to $K = 2$ and $\alpha = 5$. In the advantage-like definition of the workers' rewards (App. B.4), future signals are defined using a one-step truncation on the sequence of sub-managers' value functions (refer to Eq. 19), as empirical evidence showed better performance. App. C.4 further analyzes the impact of considering different credit assignment schemes.

The results in Fig. 3 show that HiMPPO achieves higher overall returns as the difficulty (i.e., the number of agents) increases, showing the benefits of relying on hierarchical policies to coordinate exploration in challenging scenarios. Conversely, we observed high training instability in the other hierarchical baselines (H-GPPO and H-GUPPO), further confirming the effectiveness of our novel reward-assignment mechanism.

5.3 Ablation Studies

In this section, we perform a set of ablation studies to assess the impact and effectiveness of the proposed designs. Additional experiments and analyses are reported in App. C.

Table 1: Sensitivity to different reward-assignment mechanisms in *LBFwS* and *VMAS Sampling* (avg. return $\pm$ std. of 8 and 6 runs, respectively). Training curves are reported in [App. C.6](#).

	<i>LBFwS</i>			<i>VMAS Sampling</i>		
	<i>Easy</i>	<i>Medium</i>	<i>Hard</i>	3 Agents	5 Agents	7 Agents
HiMPPO	1467 $\pm$ 183	1221 $\pm$ 138	1168 $\pm$ 155	66 $\pm$ 4	102 $\pm$ 9	121 $\pm$ 16
HiMPPO-FL	1682 $\pm$ 290	768 $\pm$ 435	381 $\pm$ 12	68 $\pm$ 4	91 $\pm$ 9	117 $\pm$ 12
HiMPPO-ER	565 $\pm$ 217	446 $\pm$ 9	375 $\pm$ 14	73 $\pm$ 4	103 $\pm$ 7	117 $\pm$ 14
HiMPPO-NL	/	/	/	33 $\pm$ 6	52 $\pm$ 4	67 $\pm$ 7

Reward Assignment To further validate the effectiveness of the hierarchical reward-assignment scheme proposed in [Sec. 3.3](#), we compare HiMPPO against different variants in both *LBFwS* and *VMAS Sampling*. In particular, we consider 1) Full Local HiMPPO (HiMPPO-FL), where we expose workers to the external reward by adding it to [Eq. 5](#), and 2) Environment Reward HiMPPO (HiMPPO-ER), where workers’ rewards are based only on the external signal (the top-level manager still receives the reward in [Eq. 3](#)). Additionally, since in *VMAS Sampling* we employ a 3-level hierarchy, in this environment we also consider 3) No Local HiMPPO (HiMPPO-NL), where sub-managers do not receive the cumulative reward associated with each worker (refer to [Eq. 4](#)). The results reported in [Tab. 1](#) show that our advantage-based hierarchical reward-assignment scheme is crucial for achieving good results in complex scenarios of *LBFwS*, further confirming what we already observed when comparing our approach against hierarchical baselines. Results in the *VMAS Sampling* environment show that preventing workers from accessing the external signal and employing advantage-like rewards do not deteriorate performance and improve sample efficiency (refer to [Fig. 10](#)). Conversely, preventing sub-managers from using local rewards leads to poor results, as shown by HiMPPO-NL.

Hierarchical Structure We consider different hierarchical graph structures to investigate their impact on decision making. In particular, we compare the state-dependent dynamic hierarchical graph $\mathcal{G}_t^*$ (refer to [Sec. 3.2](#) and [App. D.1.2](#)) against: 1) Static Graph HiMPPO (HiMPPO-SG), a 3-level HiMPPO where the topology is fixed at the beginning of each episode, i.e., the $\mathcal{G}_t^* = \mathcal{G}_0^* \forall t$; 2) 2-Levels HiMPPO (HiMPPO-2L), where we use a static 2-level hierarchy with all the workers connected to a single top-level manager. The results reported in [Tab. 2](#) show that the dynamic 3-level hierarchy is crucial for achieving better performance in *VMAS Sampling*. Indeed, HiMPPO-SG and HiMPPO-2L are limited compared to HiMPPO since the hierarchical structure does not reflect the current state. Notably, HiMPPO-SG and HiMPPO-2L perform akin. This suggests that, in this environment, processing information with a deeper (but static) hierarchy does not provide additional advantages when compared to a 2-level structure.

Table 2: Ablation study for different hierarchical graphs in the *VMAS Sampling* environment (avg. return $\pm$ std. of 6 runs). Training curves are reported in [App. C.7](#).

	3 Agents	5 Agents	7 Agents
HiMPPO	66 $\pm$ 4	102 $\pm$ 9	121 $\pm$ 16
HiMPPO-SG	51 $\pm$ 8	80 $\pm$ 9	82 $\pm$ 9
HiMPPO-2L	59 $\pm$ 9	78 $\pm$ 14	96 $\pm$ 7

6 Conclusions

We introduced *Hierarchical Message-Passing Policy* (HiMPo), a novel methodology for learning hierarchies of message-passing policies in multi-agent systems. The decision-making structure of HiMPo relies on a (dynamic) multi-level hierarchical graph, which allows for improved coordination and planning by leveraging communication and hierarchical decision-making. Notably, the proposed reward-assignment method is theoretically sound, and empirical results on different benchmarks validate its effectiveness. HiMPo opens up new designs for graph-based hierarchical MARL policies.

Limitations and future work The current implementation of HiMPo uses a goal-assignment mechanism where upper levels operate at fixed time scales. Future work can build upon HiMPo to explore more advanced goal-assignment schemes. In particular, it would be interesting to develop asynchronous mechanisms, where each (sub-)manager can send commands at different time scales based on the current state. Finally, extensions to heterogeneous multi-agent systems would broaden the applicability of HiMPo.

7 Acknowledgements

This work was supported by the Swiss National Science Foundation grants No. 204061 (*HORD GNN: Higher-Order Relations and Dynamics in Graph Neural Networks*) and No. 225351 (*Relational Deep Learning for Reliable Time Series Forecasting at Scale*).

References

- [1] Akshat Agarwal, Sumit Kumar, Katia Sycara, and Michael Lewis. Learning transferable cooperative behavior in multi-agent teams. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '20, page 1741–1743, Richland, SC, 2020. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450375184.
- [2] Sanjeevan Ahilan and Peter Dayan. Feudal multi-agent hierarchies for cooperative reinforcement learning. *arXiv preprint arXiv:1901.08492*, 2019.
- [3] Stefano V. Albrecht and Subramanian Ramamoorthy. A game-theoretic model and best-response learning method for ad hoc coordination in multiagent systems. In *Proceedings of the 2013 International Conference on Autonomous Agents and Multi-Agent Systems*, AAMAS '13, page 1155–1156, Richland, SC, 2013. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450319935.
- [4] Stefano V. Albrecht, Filippos Christianos, and Lukas Schäfer. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. URL <https://www.mar1-book.com>.
- [5] Jasmine Jerry Aloor, Siddharth Nayak, Sydney Dolan, Victor L Qin, and Hamsa Balakrishnan. Towards cooperation and fairness in multi-agent reinforcement learning. In *Coordination and Cooperation for Multi-Agent Reinforcement Learning Methods Workshop*, 2024. URL <https://openreview.net/forum?id=ze0ubBNQrA>.
- [6] Christopher Amato. An introduction to centralized training for decentralized execution in cooperative multi-agent reinforcement learning. *arXiv preprint arXiv:2409.03052*, 2024.
- [7] Adrian Arnaiz-Rodriguez and Federico Errica. Oversmoothing, "oversquashing", heterophily, long-range, and more: Demystifying common beliefs in graph machine learning. *arXiv preprint arXiv:2505.15547*, 2025.
- [8] Hicham Azmani, Andries Rosseau, Ann Nowe, and Roxana Rădulescu. Cooperative foraging behaviour through multi-agent reinforcement learning with graph-based communication. In *Sixteenth European Workshop on Reinforcement Learning*, 2023.
- [9] Davide Bacciu, Federico Errica, Alessio Micheli, and Marco Podda. A gentle introduction to deep learning for graphs. *Neural Networks*, 129:203–221, 2020.
- [10] Nikhil Barhate. Minimal pytorch implementation of proximal policy optimization. <https://github.com/nikhilbarhate99/PP0-PyTorch>, 2021.
- [11] Andrew G Barto and Sridhar Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete event dynamic systems*, 13(1):41–77, 2003.
- [12] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- [13] Matteo Bettini, Ryan Kortvelesy, Jan Blumenkamp, and Amanda Prorok. Vmas: A vectorized multi-agent simulator for collective robot learning. *The 16th International Symposium on Distributed Autonomous Robotic Systems*, 2022.
- [14] Matteo Bettini, Ajay Shankar, and Amanda Prorok. Heterogeneous multi-robot reinforcement learning. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pages 1485–1494, 2023.
- [15] Jan Blumenkamp, Steven Morad, Jennifer Gielis, Qingbiao Li, and Amanda Prorok. A framework for real-world multi-robot systems running decentralized gnn-based policies. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 8772–8778. IEEE, 2022.

- [16] Albert Bou, Matteo Bettini, Sebastian Dittert, Vikash Kumar, Shagun Sodhani, Xiaomeng Yang, Gianni De Fabritiis, and Vincent Moens. Torchrl: A data-driven decision-making library for pytorch, 2023.
- [17] Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- [18] Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Mike Rabbat, and Joelle Pineau. Tarmac: Targeted multi-agent communication. In *International Conference on machine learning*, pages 1538–1546. PMLR, 2019.
- [19] Peter Dayan and Geoffrey E Hinton. Feudal reinforcement learning. *Advances in neural information processing systems*, 5, 1992.
- [20] Christian Schroeder De Witt, Tarun Gupta, Denys Makoviichuk, Viktor Makoviychuk, Philip HS Torr, Mingfei Sun, and Shimon Whiteson. Is independent learning all you need in the starcraft multi-agent challenge? *arXiv preprint arXiv:2011.09533*, 2020.
- [21] Wei Duan, Jie Lu, and Junyu Xuan. Inferring latent temporal sparse coordination graph for multi-agent reinforcement learning. *CoRR*, abs/2403.19253, 2024.
- [22] Wei Duan, Jie Lu, and Junyu Xuan. Group-aware coordination graph for multi-agent reinforcement learning. In Kate Larson, editor, *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, pages 3926–3934. International Joint Conferences on Artificial Intelligence Organization, 8 2024. doi: 10.24963/ijcai.2024/434. URL <https://doi.org/10.24963/ijcai.2024/434>. Main Track.
- [23] Benjamin Ellis, Jonathan Cook, Skander Moalla, Mikayel Samvelyan, Mingfei Sun, Anuj Mahajan, Jakob Foerster, and Shimon Whiteson. Smacv2: An improved benchmark for cooperative multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 36:37567–37593, 2023.
- [24] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- [25] Jakob Foerster, Ioannis Alexandros Assael, Nando De Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. *Advances in neural information processing systems*, 29, 2016.
- [26] Hongyang Gao and Shuiwang Ji. Graph u-nets. In *international conference on machine learning*, pages 2083–2092. PMLR, 2019.
- [27] Mohammad Ghavamzadeh, Sridhar Mahadevan, and Rajbala Makar. Hierarchical multi-agent reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 13:197–229, 2006.
- [28] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International conference on machine learning*, pages 1263–1272. PMLR, 2017.
- [29] Nathaniel Grammel, Sanghyun Son, Benjamin Black, and Aakriti Agrawal. Revisiting parameter sharing in multi-agent deep reinforcement learning. *arXiv preprint arXiv:2005.13625*, 2020.
- [30] Daniele Grattarola, Daniele Zambon, Filippo Maria Bianchi, and Cesare Alippi. Understanding pooling in graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.
- [31] Jayesh K. Gupta, Maxim Egorov, and Mykel J. Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *AAMAS Workshops*, 2017. URL <https://api.semanticscholar.org/CorpusID:9421360>.
- [32] Eric A Hansen, Daniel S Bernstein, and Shlomo Zilberstein. Dynamic programming for partially observable stochastic games. In *AAAI*, volume 4, pages 709–715, 2004.

- [33] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
- [34] Pablo Hernandez-Leal, Michael Kaisers, Tim Baarslag, and Enrique Munoz De Cote. A survey of learning in multiagent environments: Dealing with non-stationarity. *arXiv preprint arXiv:1707.09183*, 2017.
- [35] Shengchao Hu, Li Shen, Ya Zhang, and Dacheng Tao. Learning multi-agent communication from graph modeling perspective. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Qox9r00kN0>.
- [36] Dom Huh and Prasant Mohapatra. Multi-agent reinforcement learning: A comprehensive survey. *arXiv preprint arXiv:2312.10256*, 2023.
- [37] Jiechuan Jiang and Zongqing Lu. Learning attentional communication for multi-agent cooperation. *Advances in neural information processing systems*, 31, 2018.
- [38] Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=HkxdQkSYDB>.
- [39] Weiqiang Jin, Hongyang Du, Biao Zhao, Xingwu Tian, Bohang Shi, and Guang Yang. A comprehensive survey on multi-agent cooperative decision-making: Scenarios, approaches, challenges and perspectives, 2025. URL <https://arxiv.org/abs/2503.13415>.
- [40] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.
- [41] Arbaaz Khan, Ekaterina Tolstaya, Alejandro Ribeiro, and Vijay Kumar. Graph policy gradients for large scale robot control. In *Conference on robot learning*, pages 823–834. PMLR, 2020.
- [42] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [43] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=SJU4ayYgl>.
- [44] Ryan Kortvelesy and Amanda Prorok. Qgnn: Value function factorisation with graph neural networks. *arXiv preprint arXiv:2205.13005*, 2022.
- [45] Aleksandar Krnjaic, Raul D Steleac, Jonathan D Thomas, Georgios Papoudakis, Lukas Schäfer, Andrew Wing Keung To, Kuan-Ho Lao, Murat Cubuktepe, Matthew Haley, Peter Börsting, et al. Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers. *arXiv preprint arXiv:2212.11498*, 2022.
- [46] Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. *Advances in neural information processing systems*, 29, 2016.
- [47] Vitaly Kurin, Maximilian Igl, Tim Rocktäschel, Wendelin Boehmer, and Shimon Whiteson. My body is a cage: the role of morphology in graph-based incompatible control. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=N3zUDGN510>.
- [48] Jiachen Li, Chuanbo Hua, Jinkyoo Park, Hengbo Ma, Victoria Magdalena Dax, and Mykel J. Kochenderfer. Evolvehypergraph: Group-aware dynamic relational reasoning for trajectory prediction. *CoRR*, abs/2208.05470, 2022. URL <https://doi.org/10.48550/arXiv.2208.05470>.
- [49] Sheng Li, Jayesh K Gupta, Peter Morales, Ross Allen, and Mykel J Kochenderfer. Deep implicit coordination graphs for multi-agent reinforcement learning. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, pages 764–772, 2021.

- [50] Siyuan Li, Rui Wang, Minxue Tang, and Chongjie Zhang. Hierarchical reinforcement learning with advantage-based auxiliary rewards. *Advances in Neural Information Processing Systems*, 32, 2019.
- [51] Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*, pages 157–163. Elsevier, 1994.
- [52] Chiqiang Liu and Dazi Li. HYGMA: Hypergraph coordination networks with dynamic grouping for multi-agent reinforcement learning. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=mgJkeqc685>.
- [53] Zeyang Liu, Lipeng Wan, Xue Sui, Zhuoran Chen, Kewu Sun, and Xuguang Lan. Deep hierarchical communication graph in multi-agent reinforcement learning. In *IJCAI*, pages 208–216, 2023.
- [54] Zhihao Liu, Zhiwei Xu, and Guoliang Fan. Hierarchical multi-agent reinforcement learning with intrinsic reward rectification. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023.
- [55] Xueguang Lyu, Andrea Baisero, Yuchen Xiao, Brett Daley, and Christopher Amato. On centralized critics in multi-agent reinforcement learning. *Journal of Artificial Intelligence Research*, 77:295–354, 2023.
- [56] Jinming Ma and Feng Wu. Feudal multi-agent deep reinforcement learning for traffic signal control. In *Proceedings of the 19th international conference on autonomous agents and multiagent systems (AAMAS)*, pages 816–824, 2020.
- [57] Jinming Ma and Feng Wu. Feudal multi-agent reinforcement learning with adaptive network partition for traffic signal control. *arXiv preprint arXiv:2205.13836*, 2022.
- [58] Rajbala Makar, Sridhar Mahadevan, and Mohammad Ghavamzadeh. Hierarchical multi-agent reinforcement learning. In *Proceedings of the fifth international conference on Autonomous agents*, pages 246–253, 2001.
- [59] Tommaso Marzi, Arshjot Singh Khehra, Andrea Cini, and Cesare Alippi. Feudal graph reinforcement learning. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=wFcyJTik90>.
- [60] Volodymyr Mnih. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [61] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [62] Navid Naderializadeh, Fan H Hung, Sean Soleyman, and Deepak Khosla. Graph convolutional value decomposition in multi-agent reinforcement learning. *arXiv preprint arXiv:2010.04740*, 2020.
- [63] Siddharth Nayak, Kenneth Choi, Wenqi Ding, Sydney Dolan, Karthik Gopalakrishnan, and Hamsa Balakrishnan. Scalable multi-agent reinforcement learning through intelligent information aggregation. In *International Conference on Machine Learning*, pages 25817–25833. PMLR, 2023.
- [64] Yaru Niu, Rohan R Paleja, and Matthew C Gombolay. Multi-agent graph-attention communication and teaming. In *AAMAS*, volume 21, page 20th, 2021.
- [65] Georgios Papoudakis, Filippas Christianos, Arrasy Rahman, and Stefano V Albrecht. Dealing with non-stationarity in multi-agent deep reinforcement learning. *arXiv preprint arXiv:1906.04737*, 2019.
- [66] Georgios Papoudakis, Filippas Christianos, Lukas Schäfer, and Stefano V Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021. URL <https://openreview.net/forum?id=cIrPX-Sn5n>.

- [67] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [68] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020.
- [69] Heechang Ryu, Hayong Shin, and Jinkyoo Park. Multi-agent actor-critic with hierarchical graph attention network. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7236–7243, 2020.
- [70] Mikayel Samvelyan, Tabish Rashid, Christian Schroeder De Witt, Gregory Farquhar, Nantas Nardelli, Tim GJ Rudner, Chia-Man Hung, Philip HS Torr, Jakob Foerster, and Shimon Whiteson. The starcraft multi-agent challenge. *arXiv preprint arXiv:1902.04043*, 2019.
- [71] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [72] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL <http://arxiv.org/abs/1707.06347>.
- [73] Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. Learning when to communicate at scale in multiagent cooperative and competitive tasks. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rye7knCqK7>.
- [74] Sainbayar Sukhbaatar, Rob Fergus, et al. Learning multiagent communication with backpropagation. *Advances in neural information processing systems*, 29, 2016.
- [75] Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS '18*, page 2085–2087, Richland, SC, 2018. International Foundation for Autonomous Agents and Multiagent Systems.
- [76] Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2): 181–211, 1999.
- [77] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=7UmjRGzp-A>.
- [78] Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. Feudal networks for hierarchical reinforcement learning. In *International conference on machine learning*, pages 3540–3549. PMLR, 2017.
- [79] Jiao Wang, Mingrui Yuan, Yun Li, and Zihui Zhao. Hierarchical attention master-slave for heterogeneous multi-agent reinforcement learning. *Neural Networks*, 162:359–368, 2023. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2023.02.037>. URL <https://www.sciencedirect.com/science/article/pii/S0893608023001090>.
- [80] Zhiwei Xu, Yunpeng Bai, Bin Zhang, Dapeng Li, and Guoliang Fan. Haven: hierarchical cooperative multi-agent reinforcement learning with dual coordination mechanism. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11735–11743, 2023.
- [81] Omry Yadan. Hydra - a framework for elegantly configuring complex applications. Github, 2019. URL <https://github.com/facebookresearch/hydra>.

- [82] Jiachen Yang, Igor Borovikov, and Hongyuan Zha. Hierarchical cooperative multi-agent reinforcement learning with skill discovery. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, pages 1566–1574, 2020.
- [83] Mingyu Yang, Yaodong Yang, Zhenbo Lu, Wengang Zhou, and Houqiang Li. Hierarchical multi-agent skill discovery. *Advances in Neural Information Processing Systems*, 36:61759–61776, 2023.
- [84] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35:24611–24624, 2022.
- [85] Lei Yuan, Ziqian Zhang, Lihe Li, Cong Guan, and Yang Yu. A survey of progress on cooperative multi-agent reinforcement learning in open environment. *arXiv preprint arXiv:2312.01058*, 2023.
- [86] Yihe Zhou, Shunyu Liu, Yunpeng Qing, Kaixuan Chen, Tongya Zheng, Yanhao Huang, Jie Song, and Mingli Song. Is centralized training with decentralized execution framework centralized enough for marl? *arXiv preprint arXiv:2305.17352*, 2023.
- [87] Changxi Zhu, Mehdi Dastani, and Shihan Wang. A survey of multi-agent deep reinforcement learning with communication. *Autonomous Agents and Multi-Agent Systems*, 38(1):4, 2024.

Appendix

A HiMPo

Algorithm 1 HiMPo Decision-Making Process (single step)

Require: Observations $\{z_t^w\}_{w \in \mathcal{N}}$; graph $\mathcal{G}_t^*$.

Ensure: Set of actions $\{a_t^w\}_{w \in \mathcal{N}}$.

```

1: procedure MESSAGEPASSING( $h^{\cdot,0}, \mathcal{G}^*$ )
2:   for  $l \in \{1, \dots, L_r\}$  do
3:      $h^{\cdot,l} \leftarrow \text{MP}(h^{\cdot,l-1}, \mathcal{G}^*)$  ▷ Using Eq. 1
4:   end for
5:   return  $h^{\cdot,L_r}$ 
6: end procedure

# Phase 1: Bottom-up Representation Propagation
## Phase 1a: Workers
7: for all worker  $w \in \mathcal{N}$  do ▷ in parallel
8:    $h_t^{w,0} \leftarrow \text{MLP}_w(z_t^w)$ 
9:    $h_t^{w,L_r} \leftarrow \text{MESSAGEPASSING}(h_t^{w,0}, \mathcal{G}_t^*)$ 
10: end for

## Phase 1b: Sub-managers (every  $\alpha$  steps)
11: if  $t \bmod \alpha = 0$  then
12:   for all sub-manager  $s \in \mathcal{N}_s$  do ▷ in parallel
13:      $h_t^{s,0} \leftarrow \text{AGGR}_{w \in \mathcal{C}_t^s} \{\text{MLP}_s(h_t^{w,0})\}$ 
14:      $h_t^{s,L_r} \leftarrow \text{MESSAGEPASSING}(h_t^{s,0}, \mathcal{G}_t^*)$ 
15:   end for
16: end if

## Phase 1c: Manager (every  $K\alpha$  steps)
17: if  $t \bmod K\alpha = 0$  then
18:    $h_t^m \leftarrow \text{AGGR}_s \{\text{MLP}_m(h_t^{s,0})\}$ 
19: end if

# Phase 2: Top-down Goal Generation
20: if  $t \bmod K\alpha = 0$  then ▷ Manager  $\rightarrow$  sub-managers
21:   for all sub-manager  $s \in \mathcal{N}_s$  do ▷ in parallel
22:      $g_t^{m \rightarrow s} \leftarrow \pi_\mu(h_t^m \parallel h_t^{s,0} \parallel h_t^{s,L_r})$ 
23:   end for
24: else
25:    $g_t^{m \rightarrow s} \leftarrow g_{t-1}^{m \rightarrow s}$  for all  $s \in \mathcal{N}_s$ 
26: end if

27: if  $t \bmod \alpha = 0$  then ▷ Sub-managers  $\rightarrow$  workers
28:   for all worker  $w$  (under sub-manager  $s$ ) do ▷ in parallel
29:      $g_t^{s \rightarrow w} \leftarrow \pi_\sigma(g_t^{m \rightarrow s} \parallel h_t^{s,L_r} \parallel h_t^{w,0} \parallel h_t^{w,L_r})$ 
30:   end for
31: else
32:    $g_t^{s \rightarrow w} \leftarrow g_{t-1}^{s \rightarrow w}$  for all  $w \in \mathcal{N}$ 
33: end if

# Phase 3: Worker Action Selection
34: for all worker  $w \in \mathcal{N}$  do ▷ in parallel
35:    $a_t^w \leftarrow \pi_\omega(g_t^{s \rightarrow w} \parallel h_t^{w,0} \parallel h_t^{w,L_r})$ 
36: end for
37: return actions  $\{a_t^w\}_{w \in \mathcal{N}}$ .

```

B Additional Proofs

The topology of the hierarchical graph $\mathcal{G}_t^*$ can change in time. In the execution phase, received states are aggregated according to the current topology, making nodes aware of the hierarchical decision-making structure. Nevertheless, long-term returns do not provide information concerning possible changes in $\mathcal{G}_t^*$. As such, during the training phase, nodes cannot perceive changes in the topology using the sole learning signal. For example, consider the top-level manager m : at each step $t \in T_\mu$, goals $g_t^{m \rightarrow s}$ are sent according to $\mathcal{G}_t^*$, which accounts for differences in the partitions of the underneath nodes. However, in the optimization phase, such differences do not emerge when considering the long-term return. To address this problem, during training, we assume a static hierarchy given by the topology at the first step. In other words, each trajectory is evaluated using the hierarchical graph $\mathcal{G}_0^*$ at the first step $t = 0$.

Homogeneity and weight-sharing policies As reported in [Sec. 2](#), we consider typical homogeneous settings [\[29, 4\]](#), i.e., multi-agent systems where the agents' individual dynamics are identical. This implies that the observation spaces, action spaces, and the reward function are the same. Consequently, they will also have the same expected return. For this reason, we learn a single weight-sharing policy π_ω that, for each agent, generates local (individual) actions based on local observations. Similarly, sub-managers are also homogeneous, and we denote the corresponding weight-sharing policy as π_σ . Therefore, the theoretical results can be derived for a generic worker w and sub-manager s . Note that the top-level manager m is a single agent at the highest level of the hierarchy, and it has no neighboring agents.

B.1 Proof of [Theorem 3.1](#)

Proof. Following the Markovian assumption, the probability of having a subset $\mathcal{C}_t^s$ of workers subordinate to s at time t depends only on the global state $\mathbf{S}_t$ (refer to [Sec. 3.2](#) and [App. D.1.2](#)). The top-level manager takes actions according to the current topology $\mathcal{G}_t^*$. Therefore, long-term returns achieved by sending a goal $g_t^{m \rightarrow s}$ must be evaluated by considering the rewards associated with the subset of workers $\mathcal{C}_t^s$, i.e., the objective of the manager is defined by aggregating rewards w.r.t. the initial partition of the workers. This is a reasonable assumption since the scalar learning signal (return) received by the manager does not provide any information concerning possible changes in the topology in the subsequent steps: the manager learns to provide optimal actions according to the current topology $\mathcal{G}_t^*$, and returns must be coherently defined under this setting. Given the joint policy $\pi \doteq (\pi_\omega, \pi_\sigma, \pi_\mu)$, the manager's objective reads:

$$\begin{aligned}
 \eta_m(\pi_\mu) &= \mathbb{E}_{\mathbf{S}_0} \mathbb{E}_{\mathbf{o}_0^{m \rightarrow s}} \left[V^{\pi_\mu}(\mathbf{o}_0^{m \rightarrow s}) \right] = \mathbb{E}_{\tau_m \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} r_t^{m \rightarrow s} \right] = \\
 &= \mathbb{E}_{\mathbf{S}_0} \mathbb{E}_{\mathcal{C}_0^s} \mathbb{E}_{\tau_m \sim \pi} \left\{ \sum_{t \in T_\mu} \gamma^{t/K\alpha} \text{MEAN}_{w \in \mathcal{C}_0^s} \left[\sum_{k=0}^{K\alpha-1} \bar{r}_{t+k}^w \right] \right\} = \\
 &= \mathbb{E}_{\mathbf{S}_0} \mathbb{E}_{\mathcal{C}_0^s} \left\{ \text{MEAN}_{w \in \mathcal{C}_0^s} \left[\mathbb{E}_{\tau_m \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} \sum_{k=0}^{K\alpha-1} \bar{r}_{t+k}^w \right] \right] \right\} = \\
 &= \mathbb{E}_{\mathbf{S}_0} \mathbb{E}_{\mathcal{C}_0^s} \left[\text{MEAN}_{w \in \mathcal{C}_0^s} \left\{ \eta(\pi | \mathbf{S}_0) \right\} \right] = \mathbb{E}_{\mathbf{S}_0} \left[\eta(\pi | \mathbf{S}_0) \right] = \eta(\pi) \quad (10)
 \end{aligned}$$

In the second-to-last equality, we used the homogeneity of the agents [\[4\]](#), which implies that the objective does not depend on any specific worker w as all workers will undergo the same dynamics and have the same expected return (see also [Sec. 2](#)). Therefore, the manager's objective is aligned with the optimization goal of the joint policy (refer to [Eq. 6](#)). This proves [Theorem 3.1](#). $\square$

B.2 Intermediate Results

In order to provide guarantees of alignment for the lower levels, we now report two intermediate results. To show that low-level returns are aligned with the global objective, let $\tilde{\pi}$ and $\tilde{\pi}_{i=\omega, \sigma, \mu}$ denote the old versions of π and π_i , respectively. Consider the following auxiliary term:

$$\eta_{adv}(\pi) \doteq \mathbb{E}_{\tau_m \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} A^{\tilde{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) \right] \quad (11)$$

Lemma B.1. *The objective $\eta(\pi)$ can be written in terms of the advantage over the old version $\tilde{\pi}$ of the joint policy as:*

$$\eta(\pi) = \eta(\tilde{\pi}) + \eta_{adv}(\pi) \quad (12)$$

Proof. We can write the expected return $\eta(\pi)$ of the policy π in terms of its expected advantage over the old version $\tilde{\pi}$:

$$\begin{aligned} \eta_{adv}(\pi) &= \mathbb{E}_{\tau_m \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} A^{\tilde{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) \right] = \\ &= \mathbb{E}_{\tau_m \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} [r_t^{m \rightarrow s} + \gamma V^{\tilde{\pi}_\mu}(\mathbf{o}_{t+K\alpha}^{m \rightarrow s}) - V^{\tilde{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s})] \right] = \\ &= \mathbb{E}_{\tau_m \sim \pi} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} r_t^{m \rightarrow s} - V^{\tilde{\pi}_\mu}(\mathbf{o}_0^{m \rightarrow s}) \right] = \mathbb{E}_{\tau_m \sim \pi} \sum_{t \in T_\mu} \gamma^{t/K\alpha} r_t^{m \rightarrow s} - \mathbb{E}_{\mathbf{s}_0 \mathbf{o}_0^{m \rightarrow s}} \mathbb{E}_{\tilde{\pi}_\mu} V^{\tilde{\pi}_\mu}(\mathbf{o}_0^{m \rightarrow s}) = \\ &= [\eta_m(\pi_\mu) - \eta_m(\tilde{\pi}_\mu)] = [\eta(\pi) - \eta(\tilde{\pi})] \quad (13) \end{aligned}$$

where in the last equality we used [Theorem 3.1](#). Rearranging the terms, we obtain the equality in [Eq. 12](#). $\square$

Corollary B.2. *Given [Lemma B.1](#), maximizing the global objective $\eta(\pi)$ can be written as:*

$$\max_{\pi} \eta(\pi) = \max_{\pi} \eta_{adv}(\pi) \quad (14)$$

Proof. Since $\eta(\tilde{\pi})$ is independent of the joint policy π , it is straightforward to see that maximizing $\eta(\pi)$ is independent of $\eta(\tilde{\pi})$ and the optimization of π can be expressed as [Eq. 14](#). $\square$

B.3 Proof of [Theorem 3.2](#)

Proof. As we did for the top-level manager, the return of the sub-manager s sending goal $\mathbf{g}_t^{s \rightarrow w}$ to the worker w is computed by considering the sequence of rewards obtained by w . We remark that in the $K\alpha$ steps where the top-level manager is not sending goals, the advantage function of the manager in [Eq. 4](#) is fixed for each of the K goals sent by the sub-manager. Therefore, given the joint policy $\hat{\pi}_\sigma \doteq (\tilde{\pi}_\omega, \pi_\sigma, \tilde{\pi}_\mu)$, the objective for the sub-manager s reads:

$$\begin{aligned} \eta_s(\pi_\sigma) &= \mathbb{E}_{\mathbf{s}_0 \mathbf{o}_0^{s \rightarrow w}} \mathbb{E}_{\tau_\sigma \sim \hat{\pi}_\sigma} [V^{\pi_\sigma}(\mathbf{o}_0^{s \rightarrow w})] = \mathbb{E}_{\tau_\sigma \sim \hat{\pi}_\sigma} \left\{ \sum_{t \in T_\sigma} \gamma^{t/\alpha} r_t^{s \rightarrow w} \right\} = \\ &= \mathbb{E}_{\tau_\sigma \sim \hat{\pi}_\sigma} \left\{ \sum_{t \in T_\sigma} \gamma^{t/\alpha} \left[\frac{1}{K} A^{\tilde{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) + \sum_{i=0}^{\alpha-1} \bar{r}_{t+i}^w \right] \right\} = \\ &= \mathbb{E}_{\tau_m \sim \hat{\pi}_\sigma} \left\{ \sum_{t \in T_\mu} \mathbb{E}_{\tau_s(t) \sim \hat{\pi}_\sigma} \left[\sum_{k=0}^{K-1} \mathbb{E}_{\tau_w(k) \sim \hat{\pi}_\sigma} \gamma^{t/\alpha} \gamma^k \left(\frac{1}{K} A^{\tilde{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) + \sum_{i=0}^{\alpha-1} \bar{r}_{t+k\alpha+i}^w \right) \right] \right\} \simeq \\ &\simeq \mathbb{E}_{\tau_m \sim \hat{\pi}_\sigma} \left\{ \sum_{t \in T_\mu} \frac{\gamma^{t/K\alpha}}{K} \frac{1 - \gamma^K}{1 - \gamma} A^{\tilde{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) \right\} + \mathbb{E}_{\tau_m \sim \hat{\pi}_\sigma} \left[\sum_{t \in T_\mu} \left(\gamma^{t/K\alpha} \sum_{i=0}^{K\alpha-1} \bar{r}_{t+i}^w \right) \right] = \end{aligned}$$

$$\begin{aligned}
&= \frac{1 - \gamma^K}{K(1 - \gamma)} \mathbb{E}_{\tau_m \sim \hat{\pi}_\sigma} \left\{ \sum_{t \in T_\mu} \gamma^{t/K\alpha} A^{\hat{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) \right\} + \eta(\hat{\pi}_\sigma) = \\
&= \left(\frac{1 - \gamma^K}{K(1 - \gamma)} + 1 \right) \mathbb{E}_{\tau_m \sim \hat{\pi}_\sigma} \left[\sum_{t \in T_\mu} \gamma^{t/K\alpha} A^{\hat{\pi}_\mu}(\mathbf{o}_t^{m \rightarrow s}, \mathbf{g}_t^{m \rightarrow s}) \right] + \eta(\tilde{\pi}) = k_\sigma \eta_{adv}(\hat{\pi}_\sigma) + \eta(\tilde{\pi})
\end{aligned} \tag{15}$$

where used Lemma B.1 and the approximation holds when $\gamma_\sigma \simeq \gamma \simeq 1$ and K is not too large. Following Corollary B.2, since $k_\sigma > 0$, maximizing the objective of each sub-manager $\eta_s(\pi_\sigma)$ implies maximizing the global objective $\eta(\hat{\pi}_\sigma)$. This proves Theorem 3.2. $\square$

B.4 Proof of Theorem 3.3

In general, goals received by workers are fixed for α steps. Therefore, the reward (i.e., the advantage function of the sub-manager) in Eq. 5 is constant in the α steps where the worker acts. We provide separate proofs of Theorem 3.3 for static and dynamic hierarchies to ease the presentation, despite the final result being the same. As before, we introduce the joint policy $\hat{\pi}_\omega \doteq (\pi_\omega, \hat{\pi}_\sigma, \hat{\pi}_\mu)$.

Proof - Static Hierarchies. Let us denote as s the supervisor of the worker w . The objective for w reads:

$$\begin{aligned}
\eta_w(\pi_\omega) &= \mathbb{E}_{\mathbf{s}_0 \mathbf{o}_0^w} \left[V^{\pi_\omega}(\mathbf{o}_0^w) \right] = \mathbb{E}_{\tau_w \sim \hat{\pi}_\omega} \left\{ \sum_{t=0}^{\infty} \gamma^t r_t^w \right\} = \mathbb{E}_{\tau \sim \hat{\pi}_\omega} \left\{ \sum_{t=0}^{\infty} \gamma^t \frac{A^{\hat{\pi}_\sigma}(\mathbf{o}_t^{s \rightarrow w}, \mathbf{g}_t^{s \rightarrow w})}{\alpha} \right\} = \\
&= \frac{1}{\alpha} \mathbb{E}_{\tau \sim \hat{\pi}_\omega} \left\{ \sum_{t \in T_\sigma} \gamma^t \sum_{i=0}^{\alpha-1} \gamma^i \left[r_t^{s \rightarrow w} + \gamma_\sigma V^{\hat{\pi}_\sigma}(\mathbf{o}_{t+\alpha}^{s \rightarrow w}) - V^{\hat{\pi}_\sigma}(\mathbf{o}_t^{s \rightarrow w}) \right] \right\} \simeq \\
&\simeq \frac{1 - \gamma^\alpha}{\alpha(1 - \gamma)} \mathbb{E}_{\tau \sim \hat{\pi}_\omega} \left\{ \sum_{t \in T_\sigma} \gamma^{t/\alpha} r_t^{s \rightarrow w} - V^{\hat{\pi}_\sigma}(\mathbf{o}_0^{s \rightarrow w}) \right\} = \\
&= \frac{1 - \gamma^\alpha}{\alpha(1 - \gamma)} \left\{ \mathbb{E}_{\tau \sim \hat{\pi}_\omega} \left[\sum_{t \in T_\sigma} \gamma^{t/\alpha} r_t^{s \rightarrow w} \right] - \mathbb{E}_{\mathbf{s}_0 \mathbf{o}_0^w} \left[V^{\hat{\pi}_\sigma}(\mathbf{o}_0^{s \rightarrow w}) \right] \right\} = \\
&= \frac{1 - \gamma^\alpha}{\alpha(1 - \gamma)} \left[k_\sigma \eta_{adv}(\hat{\pi}_\omega) + \eta(\tilde{\pi}) - \eta_s(\tilde{\pi}_\sigma) \right] = k_\omega \left[k_\sigma \eta_{adv}(\hat{\pi}_\omega) + \eta(\tilde{\pi}) - \eta_s(\tilde{\pi}_\sigma) \right] \tag{16}
\end{aligned}$$

where we used Theorem 3.2 and assumed $\gamma_\omega \simeq \gamma_\sigma \simeq 1$ and α not too large. Since $k_\omega > 0$ and given Corollary B.2, maximizing the objective $\eta_w(\pi_\omega)$ of each worker implies maximizing the global objective $\eta(\hat{\pi}_\omega)$. This proves Theorem 3.3 in the case of a static hierarchy. $\square$

Proof - Dynamic Hierarchies. As we did for (sub-)managers, we evaluate the trajectory by keeping the hierarchy fixed. In other words, we consider the sequence of rewards associated with the sub-manager s_0 at the first step $t = 0$. Nevertheless, defining the future rewards when using a dynamic hierarchy is an ill-posed problem. Indeed, observations $\mathbf{o}_t^{s_0 \rightarrow w}$ at future steps $t > 0$ are not properly defined when the actual supervisor δ_t of w at time t is not s_0 . However, as reported at the end of Sec. 3.3, using a 3-level hierarchy restricts the model's applicability to cooperative problems. We then expect sub-managers to be cooperative too, and we can make them benefit from the value functions of future supervisors $\delta_{t \geq 0}$ of w . Therefore, fixed the initial sub-manager s_0 and given a future step t , we can define the advantage-like reward of w under the supervision of the sub-manager s_0 as:

$$r_t^w = \frac{1}{\alpha} \left[r_t^{s_0 \rightarrow w} + \gamma_\sigma V^{\pi_\sigma}(\mathbf{o}_{t+\alpha}^{\delta_{t+\alpha} \rightarrow w}) \mathbb{I}[t + \alpha \leq t^*] - V^{\pi_\sigma}(\mathbf{o}_t^{\delta_t \rightarrow w}) \mathbb{I}[t \leq t^*] \right] \tag{17}$$

where the Iverson brackets $\llbracket \cdot \rrbracket$ allow for truncating the influence of the goal $g_0^{s_0 \rightarrow w}$ to the sequence of future sub-managers after the truncation step t^* . The objective for w reads:

$$\begin{aligned}
\eta_w(\pi_w) &= \mathbb{E}_{\mathbf{s}_0 \mathbf{o}_0^w} \mathbb{E} [V^{\pi_w}(\mathbf{o}_0^w)] = \mathbb{E}_{\tau_w \sim \hat{\pi}_w} \left\{ \sum_{t=0}^{\infty} \gamma_t^w r_t^w \right\} = \\
&= \frac{1}{\alpha} \mathbb{E}_{\tau \sim \hat{\pi}_w} \left\{ \sum_{t \in T_\sigma} \gamma_t^w \sum_{i=0}^{\alpha-1} \gamma_\omega^i \left[r_t^{s_0 \rightarrow w} + \gamma_\sigma V^{\pi_\sigma}(\mathbf{o}_{t+\alpha}^{\delta_{t+\alpha} \rightarrow w}) \llbracket t + \alpha \leq t^* \rrbracket - V^{\pi_\sigma}(\mathbf{o}_t^{\delta_t \rightarrow w}) \llbracket t \leq t^* \rrbracket \right] \right\} \simeq \\
&\simeq \frac{1 - \gamma^\alpha}{\alpha(1 - \gamma)} \mathbb{E}_{\tau \sim \hat{\pi}_w} \left\{ \sum_{t \in T_\sigma} \gamma^{t/\alpha} r_t^{s_0 \rightarrow w} - V^{\pi_\sigma}(\mathbf{o}_0^{s_0 \rightarrow w}) \right\} = \\
&= \frac{1 - \gamma^\alpha}{\alpha(1 - \gamma)} \left\{ \mathbb{E}_{\tau \sim \hat{\pi}_w} \left[\sum_{t \in T_\sigma} \gamma^{t/\alpha} r_t^{s_0 \rightarrow w} \right] - \mathbb{E}_{\mathbf{s}_0 \mathbf{o}_0^{s_0 \rightarrow w}} \mathbb{E} \left[V^{\pi_\sigma}(\mathbf{o}_0^{s_0 \rightarrow w}) \right] \right\} = \\
&= \frac{1 - \gamma^\alpha}{\alpha(1 - \gamma)} \left[k_\sigma \eta_{adv}(\hat{\pi}_w) + \eta(\hat{\pi}) - \eta_s(\hat{\pi}_\sigma) \right] = k_\omega \left[k_\sigma \eta_{adv}(\hat{\pi}_w) + \eta(\hat{\pi}) - \eta_s(\hat{\pi}_\sigma) \right] \quad (18)
\end{aligned}$$

where we used [Theorem 3.2](#) and assumed $\gamma_w \simeq \gamma_\sigma \simeq 1$ and α not too large. Since $k_\omega > 0$ and given [Corollary B.2](#), maximizing the objective $\eta_w(\pi_w)$ of each worker implies maximizing the global objective $\eta(\hat{\pi}_w)$. This proves [Theorem 3.3](#) in the case of a dynamic hierarchy. $\square$

Notice that a similar result for dynamic hierarchies can be achieved using an advantage-like reward defined in the time scale of the workers:

$$r_t^w = \frac{1}{\alpha} \left[r_t^{s_0 \rightarrow w} + \gamma_\sigma V^{\pi_\sigma}(\mathbf{o}_{t+1}^{\delta_{t+1} \rightarrow w}) \llbracket t + 1 \leq t^* \rrbracket - V^{\pi_\sigma}(\mathbf{o}_t^{\delta_t \rightarrow w}) \llbracket t \leq t^* \rrbracket \right] \quad (19)$$

where:

$$\mathbf{o}_{t+1}^{\delta_{t+1} \rightarrow w} = \begin{cases} \mathbf{o}_{t+\alpha}^{\delta_{t+\alpha} \rightarrow w} & \text{if } (t+1) \bmod \alpha = 0 \\ \mathbf{o}_t^{\delta_t \rightarrow w} & \text{otherwise} \end{cases} \quad (20)$$

By doing so, workers undergo the possible change of supervisor only at the last of the α steps. We conduct an empirical analysis of this aspect in [App. C.4](#).

C Additional Results

C.1 Comparison with Other Communication Methods

In this section, we compare HiMPPO against two additional communication methods that are not based on PPO. In particular, we consider 1) DGN [\[38\]](#), which is based on DQN [\[60, 61\]](#) and relies on attention-based graph convolutions, and 2) MAGIC [\[64\]](#), a policy gradient method that constructs dynamic agent-agent relationships and then processes the messages using a graph attention network. We compare those models on *LBFWs*, as it requires both coordination and high-level planning to be solved.

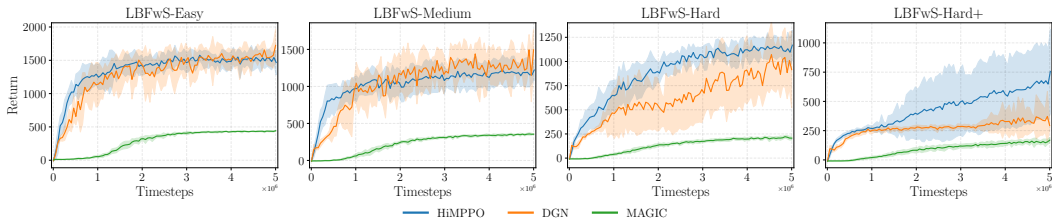


Figure 4: Results of HiMPPO, DGN, and MAGIC in the *LBFWs* environment. For each configuration, we report the average return and sample standard deviation of 8 runs.

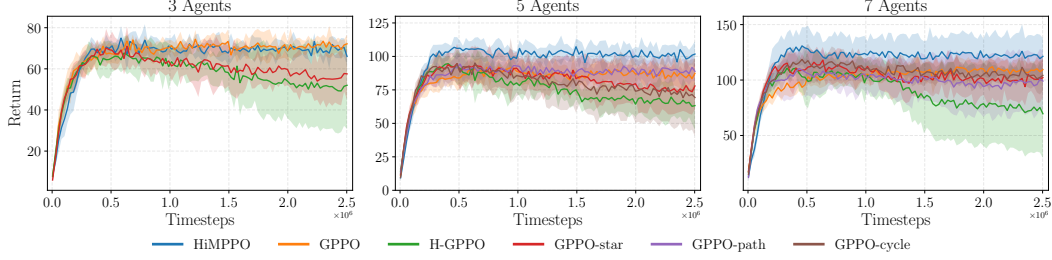


Figure 5: Results of the communication analysis in the *VMAS Sampling* environment, where we compare the performance of HiMPPO against GPPO and its variants with different communication topologies. For each configuration, we report the average return and sample standard deviation of 6 runs.

Results reported in Fig. 4 show that MAGIC always learns the individual strategy, leading to lower returns. On the contrary, DGN performs comparably with HiMPPO in *LBFwS-Easy* and *LBFwS-Medium*, while in *LBFwS-Hard* is still able to learn the cooperative strategy, even if it achieves lower average returns w.r.t. HiMPPO and is less sample efficient. Nevertheless, to further test the models’ capabilities, we consider the *LBFwS-Hard+* scenario, where the map is an 18×18 grid and there are 7 food items of level 1 and 7 food items of level 2. Here, DGN struggles to learn the cooperative strategy, while HiMPPO learns it in the majority of the runs.

C.2 Analysis of the Communication Mechanism

To analyze the impact of the proposed coordination and communication mechanism, we compare the results achieved by HiMPPO against some variants of GPPO that use a fixed graph topology. In addition to H-GPPO, which assumes an underlying fully-connected graph, we consider a star graph (GPPO-star), with a central node connected to all the others, a path graph (GPPO-path), and a cycle graph (GPPO-cycle). Note that for 3-node graphs, path and cycle graphs are equivalent to star and fully-connected graphs, respectively.

Results reported in Fig. 5 show that HiMPPO performs better than all the variants for larger systems. In particular, using fixed topologies in GPPO often hinders performance, and a fully-connected graph can make optimization unstable. Note that, globally, HiMPPO and fixed-topology variants of GPPO have access to the same information, showing that controlling how such information is processed is crucial in learning effective policies.

C.3 Sensitivity Analysis of the Goal Scale

The hyperparameters K and α represent the time steps in which (sub-)managers act, i.e., the goal-propagation frequency. In general, these time scales align decision-making to the temporal resolution of the problem, and can be set by performing a grid search. Faster scales should be adopted in tasks requiring quick adaptation to provide more frequent supervision (e.g., robotic control). Conversely, slower scales can help in temporally extended tasks, such as navigation problems.

To assess how the goal-propagation scale affects the performance of HiMPPO, we perform a sensitivity experiment of the manager’s goal scale K in *LBFwS*. In particular, in addition to $K = 5$, we consider slower ($K = 15$) and faster ($K = 2$) scales. The results in Fig. 6 show that, on average, our model is robust to the choice of K . Nevertheless, using a faster scale can effectively improve the performance in simpler tasks by providing more frequent commands, but can be detrimental in scenarios that require a coarser resolution. Indeed, because of the map size in *LBFwS-Hard*, more steps are needed to find and deliver items in this scenario.

C.4 Analysis of the Truncation Value

As reported in App. B.4, dynamic 3-level hierarchies require defining future value functions when considering the sequence of rewards associated with the sub-manager at the first step. In par-

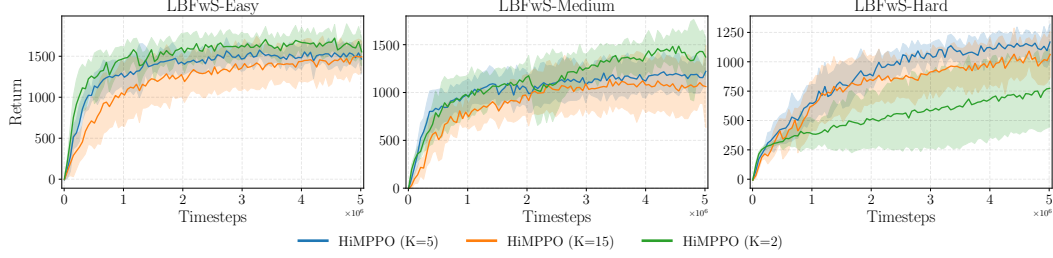


Figure 6: Results of the sensitivity analysis on the goal scale K in the *LBFwS* environment. For each configuration, we report the average return and sample standard deviation of 8 runs.

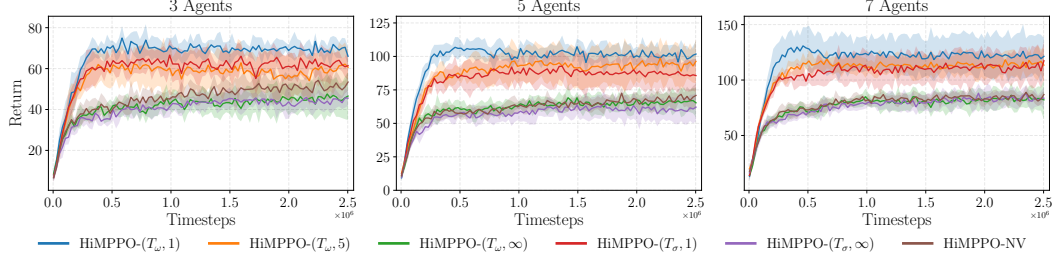


Figure 7: Results of the ablation study for different truncation schemes in the *VMAS Sampling* environment. For each configuration, we report the average return and sample standard deviation of 6 runs.

ticular, according to the desired temporal resolution, they can be defined w.r.t. the time scale of sub-managers (Eq. 17) or workers (Eq. 19). Moreover, future values $V^{\pi_\sigma}(\mathbf{o}_t^{\delta_t \rightarrow w})$ can be set to zero after a certain truncation step t^* . This threshold accounts for the degree of cooperation of the sub-managers: low values of t^* imply that the sub-manager sending the goal relies only on first estimates of the value functions, and vice versa.

To investigate these aspects, we consider both the value-assignment methods as well as different truncation values. We denote as $\text{HiMPPO}(T_\sigma, t^*)$ and $\text{HiMPPO}(T_\omega, t^*)$ the models where advantage-like rewards are defined using Eq. 17 and 19, respectively, with truncation step t^* . Furthermore, No Values HiMPPO (HiMPPO-NV) denotes the advantage-like rewards without value functions, i.e., with truncation step $t^* = 0$. The results reported in Fig. 7 show that for low t^* the model performs better. However, completely truncating the value functions from the workers' reward (HiMPPO-NV) leads to lower returns. A possible explanation could be that the sub-managers' value functions are crucial for correctly guiding the workers toward the global objective. Nevertheless, high values of t^* make credit assignment more challenging, as it considers a longer sequence of value functions, which might be related to other sub-managers.

C.5 SMACv2

We consider 3 representative maps from *SMACv2*, namely *Protoss*, *Terran*, and *Zerg*. Since hyperparameters were not tuned for this environment, we adopted the simplified 6_vs_5 scenario for all the maps. For the models leveraging graphs, we defined two allies (nodes) as connected if their distance is less than a pre-specified communication range, fixed to 3; dead allies are filtered out from the graph. For HiMPPO, the hierarchical scheme is a 2-level hierarchy where goals are propagated every 5 steps.

The results reported in Fig. 8 show that graph-based representations do not improve performance in this environment: allies' features are already encoded in the observation space of each agent, making IPPO and MAPPO the best-performing models. Indeed, independent learning can achieve remarkable performance in similar settings [20], suggesting that additional information processing mechanisms might be redundant for solving these tasks. Nevertheless, the 2-level hierarchical coordination scheme allows HiMPPO to achieve the best performance among graph-based methods. Since *SMACv2* is

a fully cooperative environment, agents receive the same reward signal at each step. This makes it even more challenging for our model to learn an effective hierarchical coordination mechanism, as goals received by the workers at the same step are associated with the same long-term return. Therefore, the manager has to learn to generate individual goals based on an implicit estimate of the local contributions of each worker. Nevertheless, despite the complex reward-assignment mechanism, HiMPPO achieves a competitive win rate percentage.

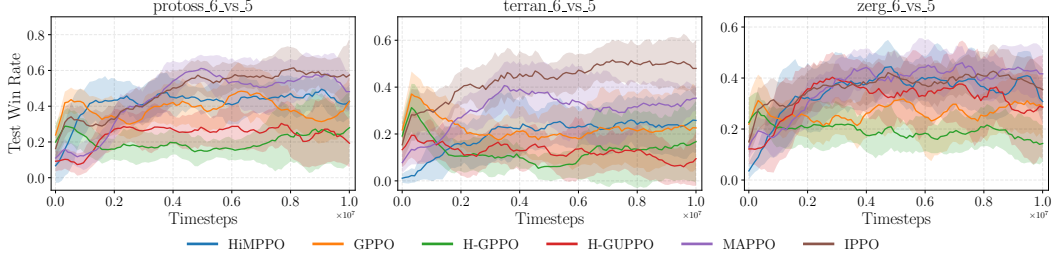


Figure 8: Results of the models for three maps of the *SMACv2* environment. We report the average test win rate and sample standard deviation of 4 runs for each configuration.

C.6 Analysis of the Reward Mechanism

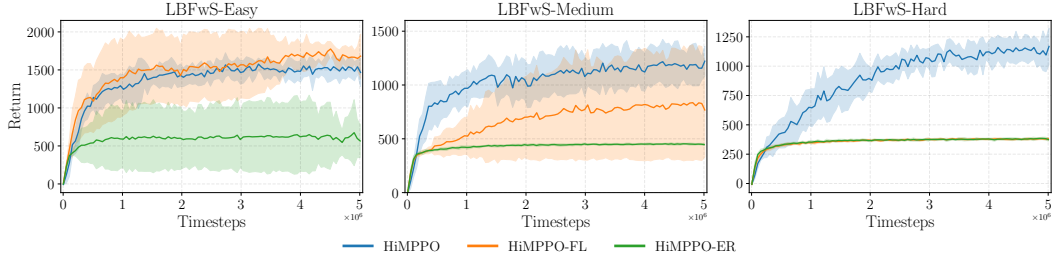


Figure 9: Results of the ablation study for different reward-assignment mechanisms in the *LBFwS* environment. For each configuration, we report the average return and sample standard deviation of 8 runs.

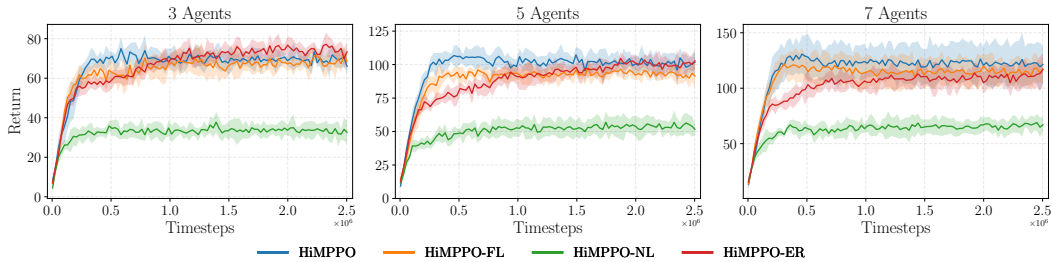


Figure 10: Results of the ablation study for different reward-assignment mechanisms in the *VMAS Sampling* environment. For each configuration, we report the average return and sample standard deviation of 6 runs.

C.7 Analysis of the Hierarchical Structure

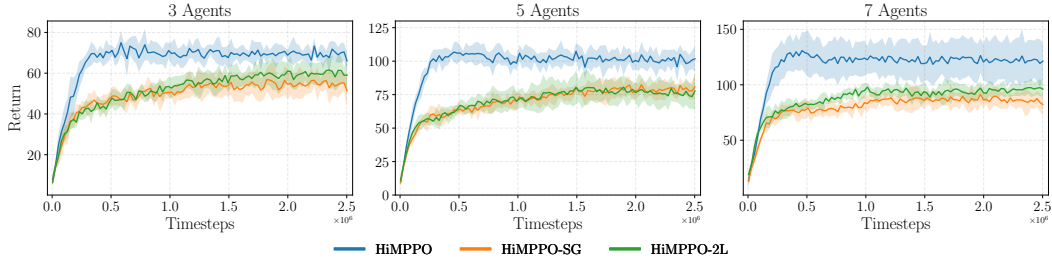


Figure 11: Results of the ablation study for different hierarchical graphs in the *VMAS Sampling* environment. For each configuration, we report the average return and sample standard deviation of 6 runs.

D Experimental Setting

D.1 Environments

D.1.1 Level-Based Foraging with Survival (LBFwS)

Starting from the codebase provided by Azmani et al. [8], we modify the Level-Based Foraging [3, 66] environment by further exacerbating the cooperative-competitive behavior of the agents to make it more challenging. In particular, once the food is within its range, an agent can eat it or deliver it to a landmark. Following the original version, the first strategy leads to an immediate, individual reward. Conversely, delivering food extends the episode duration: even if it gives no individual reward in the short term, it is beneficial for the team of agents, allowing them to have more time to collect food and achieve a potentially higher cumulative return. This behavior mirrors community food-sharing dynamics, where individuals contribute to the group’s survival. This extended version of the environment, named *Level-Based Foraging with Survival* (LBFwS), aggravates the cooperative-competitive dynamics: in addition to evenly dividing the reward when cooperating [8], trying to deliver food can be harmful to the agent, as other agents might act greedily to maximize their individual reward instead of adopting the group’s survival strategy. Furthermore, delivering food requires a complex and temporally extended series of actions, i.e., collecting the item, transporting it to the landmark, and releasing it, making the problem challenging in terms of exploration and credit assignment. We now illustrate the differences of the LBFwS environment with respect to the version of Azmani et al. [8].

Observation Space Compared with the original setting, each agent has an additional grid representing its local observation of the landmark layer, i.e., if the landmark is within its sight range and if the agent (or its neighbors) is carrying food. Furthermore, each i -th agent observation is augmented by a 4-dimensional vector $(t, t_s, x_{lm}^i, y_{lm}^i)$, where t is the time passed so far (normalized with the absolute maximum number of steps T), t_s is the survival time (which can be incremented by delivering food) normalized with the survival initial value $T_\sigma < T$, and x_{lm}^i and y_{lm}^i are the normalized relative displacements in the (x, y) axis w.r.t. the landmark position. Therefore, agents are always aware of the position of the landmark, which is static and placed in the middle of the map.

Action Space We add two possible actions to the original 6-dimensional discrete action space, i.e., PICK and DELIVERY. Agents that take the EAT action while carrying food consume the item. Conversely, if an agent attempts to pick up food while already carrying an item, that action is automatically canceled. In general, unsuccessful pickup or delivery attempts do not affect the current

state of the environment. We remark that items of higher value need the cooperation of multiple agents to be consumed.

Reward We highlight that the reward structure does not change, as delivering food does not affect the agents’ rewards.

Transition Dynamics The survival and absolute counters start at $T_\sigma = T_\sigma$ and $t = 0$, respectively. When an agent successfully delivers food to the landmark, the counter T_σ is increased by the value of the food, incremented by a factor k_{surv} . At each step, the survival (absolute) counter decreases (increases) by one, and the episode terminates either when $T_\sigma = 0$ or $t = T$. Each agent can carry only one item, while unsuccessful deliveries do not imply negative rewards or changes in the item that is carried. Furthermore, an agent can decide to eat the food that is being carried.

Experimental Setup In our experiments, we fix the maximum values for survival and absolute steps as $T_\sigma = 100$ and $T = 500$, respectively. Furthermore, we set the constant for the survival time increment to $k_{surv} = 10$: as an example, successfully delivering an item with value 4 leads to an increment of T_σ of 40 time steps. In our experiments, we consider three different maps of different sizes and amounts of resources. In particular, we consider: 1) *LBFwS-Easy*, where the map is a 9×9 grid and there are 4 food items of level 1 and 4 food items of level 2; 2) *LBFwS-Medium*, where the map is a 12×12 grid and there are 5 food items of level 1 and 5 food items of level 2; 3) *LBFwS-Hard*, where the map is a 15×15 grid and there are 6 food items of level 1 and 6 food items of level 2. We remark that level 2 items need the cooperation of at least 2 agents to be picked up. For all the scenarios, the number of agents is set to 10. Despite having more food available when the difficulty increases, the grid size makes exploration and temporal credit assignment more challenging. Furthermore, this difficulty is compounded by the sparser distribution of food items in the space, which makes successful deliveries to the landmark more complex.

D.1.2 Hierarchical Graph Generation in Sampling

As aforementioned, in the *VMAS Sampling* scenario, we consider a 3-level dynamic graph $\mathcal{G}_t^*$ for the HiMPPO model. This graph is built according to the current state $\mathbf{S}_t$ of the environment. In particular, the grid is divided into 4 quadrants, and each worker (agent) is assigned to a single sub-manager based on its position. Therefore, the set $\mathcal{N}_s$ of sub-managers is composed of 4 node indexes, one for each quadrant. Refer to Fig. 12 for a visual example. We augment the initial representation $\mathbf{h}_t^{s,0}$ of each sub-manager s with a one-hot vector to give an explicit indexing to the sub-managers. However, we remark that this does not prevent the transferability of the architecture to systems with a different number of agents. There might be states where a sub-manager has no workers underneath, e.g., the up-right (orange) sub-manager of Fig. 12. In such cases, the top-level manager sends a goal to account for possible future changes in the dynamics, but the corresponding trajectory is discarded in the learning procedure.

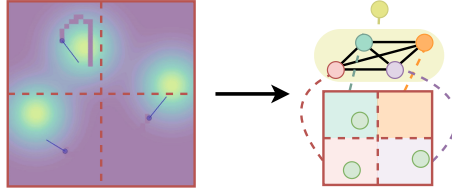


Figure 12: Building the 3-level hierarchical graph $\mathcal{G}_t^*$ at time t for the *VMAS Sampling* scenario.

D.2 Implementation Details

To achieve the optimal configurations, for all the models we performed a grid search on *LBFwS-Medium* environment, focusing mainly on latent dimensions, hidden units, and learning rates. For vector-based observations, encoders are designed as a linear layer. For grid-based observations (*LBFwS*), we use a Convolutional Network, where we tuned the number of output features and kernel size; then, two linear encoders process the flattened space and time observations to obtain the

desired latent dimension. All the models use a discount factor $\gamma = 0.99$ and the Adam [42] optimizer. For PPO-based models, the entropy coefficient and clipping values are fixed to 0.01 and 0.2, respectively. Advantages are approximated using Generalized Advantage Estimator [71] $GAE(\gamma, \lambda)$: λ is set to 0.95, except for upper levels of HiMPPO (i.e., (sub-)managers) and H-GUPPO, that is fixed to 0. For the *VMAS Sampling* environment, we tuned the initial action standard deviation, which then decays by 0.05 every $2.5 \cdot 10^5$ steps up to 0.1, except for H-GUPPO where it decays every 10^6 steps. Policies are updated for 30 epochs with a batch size of 128 every 40 (*LBFwS* and *SMACv2*) and 32 (*VMAS Sampling*) episodes. Unless constrained by the action space, we use ReLU as the activation function. For HiMPPO and GPPO, the agents' message function is implemented as an MLP with 64 hidden units in *LBFwS* and *SMACv2*, and as a Graph Convolutional Network (GCN) [43] in *VMAS Sampling*. When employing a 3-level hierarchy (*VMAS Sampling*), HiMPPO uses an MLP with 64 hidden units as the message function for sub-managers. DGN uses the same number of epochs and batch size, but updates the model at each episode and has a buffer of capacity 10^5 .

HiMPPO All the hyperparameters are shared between the different levels of the hierarchy, except for λ . We used a learning rate of 0.0001 and 0.0005 for the actor and critic, respectively. To keep the number of parameters bounded, all the functions are implemented as linear layers, except for the message function that has 64 hidden units; we consider 1 round of message passing. Embedding and goal dimensions are fixed to 64. For (sub-)managers, encoded features of subordinate nodes are aggregated using the sum. The goal is sampled from a Gaussian policy with an initial standard deviation of 0.5. For the *VMAS Sampling* environment, where continuous actions are required, we use the same value for the initial standard deviation. For *LBFwS*, we consider a 2-dimensional convolution with 8 output channels and kernel size 2. To ensure stability among different hierarchy levels, we clip gradients to the range $[-1, 1]$.

GPPO This model does not share modules between the actor and critic. The actor and critic have a hidden dimension of 64 and 32, respectively. We used a learning rate of 0.0001 for both. The embedding dimension is fixed to 64. We perform 2 message-passing rounds. For the *VMAS Sampling* environment, the initial standard deviation is set to 0.5. For *LBFwS*, we consider a 2-dimensional convolution with 32 output channels and kernel size 2. Gradients are clipped to the range $[-5, 5]$.

H-GUPPO In this model, the embedding is trained together with the actor, while the critic uses the last representation to get the value function. The actor and critic have a hidden dimension of 64 and 128, whereas the learning rates have values 0.0005 and 0.0001, respectively. The embedding dimension is fixed to 128. The depth of the U-Net is fixed to 2. For the *VMAS Sampling* environment, the initial standard deviation is set to 0.5. For the *VMAS Sampling* environment, the initial standard deviation is set to 0.8. For *LBFwS*, we consider a 2-dimensional convolution with 32 output channels and kernel size 2. Gradients are clipped to the range $[-5, 5]$.

MAPPO In this baseline, the embedding dimension is 128. The actor has 128 hidden units, whereas the critic is implemented as a linear layer. We used a learning rate of 0.0005 and 0.0001 for the actor and critic, respectively. For the *VMAS Sampling* environment, the initial standard deviation is set to 0.5. For *LBFwS*, we consider a 2-dimensional convolution with 8 output channels and kernel size 2. Gradients are clipped to the range $[-5, 5]$.

IPPO This model uses an embedding dimension of 64. The actor and critic have 128 and 64 hidden units, whereas the learning rates have values 0.0005 and 0.00001, respectively. For the *VMAS Sampling* environment, the initial standard deviation is set to 0.8. For *LBFwS*, we consider a 2-dimensional convolution with 8 output channels and kernel size 2. Gradients are clipped to the range $[-5, 5]$.

DGN For this baseline, we use a learning rate of 0.0001 and a latent dimension of 128. We use a hidden layer of 128 units and follow the original implementation for the attention model. For *LBFwS*, we consider a 2-dimensional convolution with 16 output channels and kernel size 2. We perform 100 warm-up episodes, while the target network is updated every 10 episodes; the exploration coefficient decays from 0.9 to 0.1 with a decay rate of 0.0001 per episode.

MAGIC In this model, we use a learning rate of 0.001 and a latent dimension of 256. The model learns directed communication graphs with self-loops. The two graph attention networks (sub-

processors) have 2 and 4 heads, with a hidden size of 32, and do not use normalization. We learn the first communication graph, while the second one is set as complete. For *LBFwS*, we consider a 2-dimensional convolution with 32 output channels and kernel size 2.

D.3 Hardware and Software

The code for the model and the baselines was developed in *Python 3.10* and made use of *NumPy* [33], *PyTorch* [67], and *PyTorch Geometric* [24]. We implemented the PPO-based models starting from a public repository [10] (MIT License), while we adapted the official implementations for both DGN¹ and MAGIC² to our setting. For the environments, we used the official repositories [13, 8, 23] and the utility functions of *TorchRL* [16] (MIT License). *VMAS Sampling* uses a GPL3.0 License, while *SMACv2* uses MIT License. Experiment configurations were managed using *Hydra* [81] and tracked using *Neptune.ai*³. Simulations were run on two workstations equipped with AMD EPYC 7513 and Intel Xeon E5-2650 CPUs. Depending on the model, training times take 2 – 10 hours for *VMAS Sampling*, 8 – 15 hours for *LBFwS*, and 1 – 20 days for *SMACv2*.

¹https://github.com/jiechuanjiang/pytorch_DGN (MIT License)

²<https://github.com/CORE-Robotics-Lab/MAGIC> (MIT License)

³<https://neptune.ai/>